Breaking the Combinatorial Explosion: Graph Neural Networks Guided Pruning for Optimal Bundle Pricing

SUBMISSION 364

Bundle pricing refers to designing several product combinations (i.e., *bundles*) and determining their prices in order to maximize the expected profit. It is a classic problem in revenue management and arises in many industries, such as e-commerce, tourism, and video games. However, the problem is typically intractable due to the exponential number of candidate bundles. In this paper, we explore the usage of graph neural networks (GNNs) in solving the bundle pricing problem. Specifically, we first develop a graph representation of the mixed bundling model (where every possible bundle is assigned with a specific price) and then train a GNN to learn the latent patterns of optimal bundles. Based on the trained GNN, we propose two inference strategies to derive high-quality feasible solutions. A local-search technique is further proposed to improve the solution quality. Numerical experiments validate the effectiveness and efficiency of our proposed GNN-based framework. Using a GNN trained on instances with 10 products, our methods consistently achieve over 96% of the optimal profit with only a fraction of computational time for problems of small or medium size. It also achieves superior solutions for larger size of problems compared with other heuristic methods such as bundle size pricing (BSP). We also propose an Iterative Self-Improvement strategy to further enhance the model's scalability across number of products. This strategy can provide high quality solutions for instances with 100 products even for the challenging cases where product utilities are non-additive.

Contents

Manuscript submitted for review to the 27th ACM Conference on Economics & Computation (EC'26).

1 Introduction

Bundle pricing is a widely adopted strategy across industries such as e-commerce, tourism, digital subscriptions, and retail. It refers to the practice that a firm provides combinations (i.e., "*bundles*") of products or services (at discounted prices), supplementing the traditional component pricing (CP) strategy where products are only sold separately, each with its own price. For instance, Amazon Prime combines fast shipping, video streaming, and music services into a single package, and reports show that subscription bundles drive $44 billion in subscription revenue for Amazon in 2024 [Modern Retail, 2024]. Similarly, Statista estimates a significant growth of the global digital subscription market, estimated at $832.99 billion in 2023 and projected to reach $1902.28 billion in 2030. This expansion is largely fueled by the increasing adoption of bundled subscription packages, such as Amazon Prime's combination of fast shipping, video streaming, and music services, and Disney+'s bundled offerings with Hulu [Grand View Research, 2023]. To implement this strategy effectively, a firm must solve a complex optimization problem: determining which subsets of products to offer and setting their corresponding prices to maximize total profit, under the constraint that heterogeneous customers will self-select the options that maximize their own surplus.

The bundle pricing problem is inherently combinatorial: with n products, the number of possible bundles grows exponentially (2^n). Classical formulations, such as the mixed bundling (MB) model by Hanson and Martin [1990], rigorously capture customer surplus constraints but become computationally intractable as n increases (e.g., greater than 15). Later approximations, such as bundle-size pricing (bundles with the same sizes share the same prices), see, e.g., Chu et al. [2011], improve scalability but rely on strong simplifying assumptions (e.g., ignoring product heterogeneity), limiting their realism.

It is crucial to emphasize that our work addresses the challenging non-additive valuation setting, where consumer valuations and costs for bundles exhibit sub-additivity or super-additivity. While the mixed bundling MILP formulation for deterministic valuations appears syntactically identical regardless of additivity, the computational complexity diverges fundamentally. As established by Hanson and Martin [1990] in their analysis of pricing large numbers of components, strict additivity allows for problem decomposition. When valuations are additive, the optimal bundle price is simply the sum of individual item prices, collapsing the exponential search space into a linear problem of pricing individual items independently. In contrast, the non-additive setting breaks this independence, enforcing a tightly coupled combinatorial structure where the price of a bundle cannot be inferred from its components.

In recent years, neural networks have demonstrated remarkable success in "Learning to Optimize" (L2O) for combinatorial problems, such as vehicle routing [Kool et al., 2019, Nazari et al., 2018] and MILP solving via neural branching [Gasse et al., 2019] or neural diving [Nair et al., 2020]. Motivated by these advances, one might consider leveraging these neural-network-based MILP solvers to accelerate the solution process. However, they share a fundamental dependency on the explicit graph representation of the MILP's variable-constraint matrix. In the bundle pricing context, the number of potential bundles 2^n leads to a catastrophic memory bottleneck. For any medium/large product size n, the resulting bipartite graph becomes too massive to be stored or processed by a GNN, rendering these neural heuristics computationally infeasible at the very stage of model initialization.

To overcome these challenges, we investigate how to design a scalable GNN framework that learns to identify high-quality bundle structures directly from data, without constructing the explicit exponential graph. Specifically, we aim to address the following research questions:

(1) ***Representation:*** *How can we leverage graph neural networks to extract features from the bundle pricing problem without explicitly constructing the exponentially large variable-constraint graph?*
(2) ***Search Space Reduction:*** *Given the neural network predictions, how can we design inference policies that effectively prune the intractable 2^n search space into a manageable subset?*
(3) ***Scalability:*** *How does the performance of the neural-network-based approach scale with problem size, particularly in the challenging non-additive setting?*

To address the first research question, we propose a GNN-based framework that learns to identify the latent structure of the bundle pricing problem and hence provides high-quality solutions. Leveraging the structural properties of this problem, we construct a bipartite graph representing these customer-product interactions, which scales linearly with the number of segments and products. We then train a GNN on this compact representation to extract latent structural features characterizing customer-product affinities. These features are decoded to predict selection probability matrix, representing the probability that each customer segment will purchase each product in the optimal solution. This approach effectively learns the latent mapping between problem parameters and optimal bundle structures without suffering from memory bottlenecks.

To address the second research question, we develop two inference strategies that utilize the GNN's predicted probabilities to generate candidate solutions. The first inference policy is the *Fixed Cutoff Pruning* (FCP) policy. For each customer segment, this policy first filters out products with low predicted purchase probabilities to form a segment-specific candidate bundle. The overall search space is then constructed by taking the union of these candidate bundles across all segments, reducing the total bundle count from 2^n to at most m. The second inference policy is the *Progressive Cutoff Pruning* (PCP) policy. For each customer segment, this policy ranks products by their predicted purchase probabilities to construct a sequence of nested candidate bundles. The overall search space is then formed by taking the union of these nested sets across all segments. Based on the pruned space, we apply Hanson and Martin [1990]'s mixed integer linear programming (MILP) formulation to find the optimal prices, which successfully maintain near-optimal profit performance while saving significant computational costs. FCP policy reduces the number of potential bundles from 2^n to at most m, and PCP policy reduces the number of potential bundles to at most $mn + 1$. This dramatic reduction transforms an otherwise intractable exponential problem into a computationally efficient formulation. To further refine solution quality, we incorporate a local search method that strategically adds or drops products, guided by the probability predictions of our GNN model.

To address the third research question, we evaluate the cross-scale generalization capability of our framework. Specifically, the GNN's weight-sharing capability enables us to train the model on tractable small-scale instances (where optimal labels are efficiently computable) and directly apply it to intractable large-scale ones. We conduct extensive experiments on synthetic datasets with non-additive valuations. Our results demonstrate that for small-scale problems ($n \leq 10$), our strategies consistently achieve over 96% of the optimal profit with a fraction of the runtime required by baselines. For medium-to-large scale problems ($n \geq 15$) where mixed bundling is computationally intractable, our approach consistently outperforms heuristics like bundle-size pricing, providing a high-quality scalable solution to the non-additive bundle pricing problem. We also propose an Iterative Self-Improvement strategy to further enhance the model's scalability across the number of products.

2 Related Work

This work is broadly related to three streams of research: bundle pricing, neural-network-based optimization, and bundle recommendation.

Bundle pricing. The study of bundle pricing has a long history in economics and operations research communities. Early work in the two-product setting established the profitability of bundling relative to separate sales: Stigler [1963] first shows that pure bundling (PB) in which products are only sold together as a package at a single price can dominate component pricing in profitability for two products, while Adams and Yellen [1976] and Schmalensee [1984] demonstrate the profitability for CP, PB and MB for the two-products case. Later research extends these insights to large-scale settings. A stream of work analyzes simplified mechanisms such as CP (Abdallah 2019), PB (Abdallah 2019, Bakos and Brynjolfsson 1999), and bundle-size pricing (BSP) [Chu et al., 2011]. Specifically, Bakos and Brynjolfsson [1999] prove that PB approximates the revenue of MB when the number of products grows large under zero costs and independent and additive valuations. Later, Abdallah [2019] provides lower bounds for revenue loss under positive costs from PB. Empirical and theoretical studies show that BSP can outperform CP and PB in certain conditions and can achieve asymptotic optimality when product costs are homogeneous, though it deteriorates under cost heterogeneity [Abdallah et al., 2021, Chu et al., 2011, Hitt and Chen, 2005, Li et al., 2021]. More recently, novel mechanisms have been proposed, such as pure bundling with disposal for costs [Ma and Simchi-Levi, 2021], lootbox schemes [Chen et al., 2021], and component pricing with bundle-size discounts (CPBSD) [Chen et al., 2022], which unify CP, PB and BSP approaches and achieve constant-factor guarantees. Complementary to mechanism design, a computational line of work has modeled bundle pricing directly as a mixed-integer program. The seminal work by Hanson and Martin [1990] pioneers the MB formulation of the optimal bundle pricing problem, showing that it preserves rigorous surplus maximization and price sub-additivity constraints, though its scalability is limited by the exponential growth of candidate bundles. Our work follows this computational tradition: we retain Hanson and Martin [1990]'s framework while leveraging graph-based learning to prune the bundle space before invoking the MILP solver, thereby combining theoretical rigor with practical scalability.

Neural-network-based optimization. Recently, the use of machine learning to accelerate the solution of combinatorial optimization problems has attracted increasing attention. A seminal work by Gasse et al. [2019] encodes mixed-integer programs as constraint-variable bipartite graphs and trains a GCN to imitate strong branching decisions, accelerating branch-and-bound (B&B) and inspiring a series of follow-up work. For example, Nair et al. [2020] introduce neural diving and neural branching strategies for B&B; Ding et al. [2020] extend the bipartite representation to a tripartite graph to predict partial solutions; Gupta et al. [2022] propose a lookback imitation framework for branching efficiency; Sonnerat et al. [2021] combine neural diving with neural neighborhood search to improve solution quality, and Paulus and Krause [2023] design GCN-based diving methods to facilitate B&B. In the context of linear programming, Fan et al. [2023] propose GCN-based initial basis selection to warm-start simplex and column generation, while Liu et al. [2024] train GCNs to imitate expert pivot policies, shortening pivot paths. These studies confirm that GCNs can substantially reduce computational effort in solving optimization problems. In addition to accelerating optimization solvers, machine learning is also directly integrated into the core structure of operations research problems. Kool et al. [2019] introduce an attention-based model that learns powerful heuristics for complex routing problems like the traveling salesman problem (TSP) and the vehicle routing problem (VRP), achieving near-optimal results. Nazari et al. [2018] utilize reinforcement learning to Guo et al. [2025] integrate first-order methods with a self-supervised neural network framework to efficiently solve large-scale, constrained assortment optimization problems across various complex choice models. Li et al. [2025] leverage the generalizability of GCNs to solve large-scale NP-hard constrained assortment optimization problems under the mixed multinomial logit choice model. In a similar vein, we apply GNNs directly to the bundle pricing problem, where the GNN learns segment–bundle interactions and predicts promising candidates. In

this way, our approach preserves the theoretical rigor of Hanson and Martin [1990]'s model while leveraging GNN predictions to accelerate large-scale instances, thereby extending GNN-assisted optimization to a core revenue management setting.

Bundle Recommendation. Parallel to the economics and operations research literature on pricing, the problem of bundle recommendation has gained significant traction within the computer science community. However, it is crucial to distinguish the fundamental objectives and constraints of these two streams. Bundle pricing, rooted in the seminal work of Hanson and Martin [1990], is cast as a supply-side optimization problem. It seeks to construct an optimal menu, determining specific prices to maximize total profit, subject to the strict constraint that customers self-select the option maximizing their surplus. This requirement introduces a complex coupling absent in recommendation tasks: the price of one bundle directly alters the feasibility and demand of all others. Consequently, the primary challenge lies in the combinatorial explosion of pricing constraints rather than the learning of preferences.

In contrast, bundle recommendation is primarily a demand-side prediction problem. Its goal is to infer latent user preferences from sparse historical interactions (e.g., clicks or purchases) to generate a personalized ranking of pre-existing bundles. Here, price is often treated as a static attribute or context rather than a core decision variable. This distinction is reflected in the evaluation metrics: while recommendation systems are typically assessed using ranking metrics such as Precision, Recall, and NDCG (Normalized Discounted Cumulative Gain), bundle pricing solutions are evaluated based on their economic performance (total profit) and computational efficiency. For instance, within the recommendation domain, Sar Shalom et al. [2016] propose a two-layered collaborative filtering framework designed to optimize item list recommendations by maximizing the click-through rate.

Notably, Beladev et al. [2016] bridge these two domains. Unlike standard recommender systems that focus solely on ranking accuracy (e.g., recall or NDCG), Beladev et al. [2016] integrate a personalized pricing mechanism into the recommendation process. They utilize collaborative filtering not just to maximize the customers' probability of purchase but also to maximize revenue gained from selling the bundle by predicting bundle prices. However, their approach assumes a personalized pricing scheme where sellers tailor bundles and prices for individual customers. This differs fundamentally from our setting of market-wide bundle pricing, where the seller must construct a single, consistent menu of bundles available to all customers simultaneously. Following this early exploration, recent advancements have shifted focus towards capturing complex user-bundle-item relations using deep learning techniques. Recent advancements have increasingly leveraged Graph Neural Networks (GNNs) to explicitly model the complex high-order dependencies between users, bundles, and items. For instance, Chen et al. [2019] propose a Deep Attentive Multi-Task model (DAM) that aggregates item embeddings via attention mechanisms while jointly modeling user-bundle and user-item interactions to alleviate sparsity. Chang et al. [2021] introduce the Bundle Graph Convolutional Network (BGCN), which unifies user-item and user-bundle interactions into a heterogeneous graph to capture item-level semantics and bundle-level associations. Further improvements include incorporating cross-view contrastive learning to align user-bundle and user-item representations [Ma et al., 2022], and employing diffusion models to denoise interaction graphs for robust preference learning [Li, 2025]. Specifically, these methods fall short in addressing the pricing dimension. Models like DAM [Chen et al., 2019] and BGCN [Chang et al., 2021] treat price merely as a fixed input parameter, implicitly absorbing it into static latent embeddings without the capacity for optimization. Conversely, recent advancements such as Ma et al. [2022] and Li [2025] completely disregard price information. Consequently, it is fundamentally difficult to derive a valid pricing vector for their recommendations that satisfies the strict surplus maximization constraints required in a market-wide bundle pricing.

3 Problem Definition

We study a bundle pricing problem where a seller offers n distinct products [1] to m heterogeneous customer segments. Hence, there are a total of 2^n bundle choices, denoted as $\mathcal{B} = \{0, 1, \ldots, L\}$, where $L = 2^n - 1$ and 0 indicates the empty bundle. The seller chooses the set of prices $\{p_b \mid b \in \mathcal{B}\}$ for bundles to maximize profit, which can be defined as $\sum_{k=1}^{m} \sum_{b \in \mathcal{B}} (p_b - c_{kb}) \cdot \theta_{kb}$, where c_{kb} is the serving cost of selling bundle b to segment k and $\theta_{kb} \in \{0, 1\}$ denotes whether segment k selects bundle b under self-selection assumption.

The demand for bundles follows standard assumptions: each segment k has a certain valuation R_{kb} for bundle b. After observing the price p_b, each buyer would select exactly one bundle that maximizes his/her individual surplus, defined by $s_{kb} = R_{kb} - p_b$, provided that the surplus is non-negative. We follow the assumption of free disposal of unwanted products within a bundle [Hanson and Martin, 1990]. Thus, price monotonicity B.1 is maintained.

Under these assumptions, the seller's profit optimization problem can be formulated as follows:

$$\max \quad \sum_{k=1}^{m} \sum_{b \in \mathcal{B}} (p_b - c_{kb}) \cdot \alpha_k \, \theta_{kb} \tag{1}$$

$$\text{s.t.} \quad \theta_{kb} \leq \mathbb{1}\left(b \in \arg\max_{b' \in \mathcal{B}} \{R_{kb'} - p_{b'}\} \ \wedge \ R_{kb} - p_b \geq 0 \right), \quad \forall k, b \in \mathcal{B} \tag{2}$$

$$\sum_{b \in \mathcal{B}} \theta_{kb} = 1, \quad \forall k \tag{3}$$

$$p_b \geq 0, \quad \forall b \in \mathcal{B}. \tag{4}$$

Here, α_k is the proportion of customer segment k, and $\mathbb{1}(\cdot)$ is the indicator function that enforces buyers' surplus maximization choice.

In the following discussion, we assume $R_{kb} = f\left(\sum_{j \in b} u_{kj}\right)$ where u_{kj} is the utility of product j for segment k, $f(\cdot)$ is an increasing concave function, capturing potential correlations among products. This setting is consistent with Abdallah et al. [2021]. We assume $c_{kb} = \left(\sum_{j \in b} c_j^u\right) + c_k^s$, where c_k^s is the cost associated with serving customer segment k (e.g., the shipping cost) and c_j^u is the unit cost of product j. The concavity of $f(\cdot)$ captures the economic consensus of diminishing marginal utility, and the cost structure captures the principle of economies of scale: since the fixed cost is spread across all items in the bundle, the average cost per item decreases as more products are added. Hanson and Martin [1990] formulated this problem as a mixed-integer linear program (MILP). For the sake of completeness, we describe this formulation in the following.

Remark 1. *While strict additivity facilitates problem decomposition by collapsing the search space into a linear problem using composite bundles, requiring only mn product utilities to solve the problem, general non-additive valuations present a significantly greater challenge. In the absence of additivity, the value of a bundle cannot be derived simply by summing the prices of its constituents. Consequently, instead of storing just n values, one must account for the valuations of all $m \cdot 2^n$ possible bundles in the MILP. This combinatorial explosion expands the input size and search space from linear to exponential, rendering the optimization problem computationally intractable.*

3.1 Mixed Bundling Formulation [Hanson and Martin, 1990]

Consider the problem with all possible bundles denoted by $\mathfrak{B}$. We define the following variables: θ_{kb} is a binary indicator for whether segment k chooses bundle b; p_b is the price of bundle b; P_{kb} is

[1] Scenarios involving repeated products can be addressed by adding copies of that product to the problem.

the effective price paid by segment k; s_k is the surplus of segment k; Z_{kb} is the profit from assigning bundle b to segment k; and A_b is the set of components which define bundle b. The mixed bundle pricing problem can be written as follows:

$$\max \quad \sum_{k=1}^{m} \sum_{b \in \mathfrak{B}} \alpha_k \cdot Z_{kb} \tag{5}$$

$$\text{s.t.} \quad \sum_{b \in \mathfrak{B}} \theta_{kb} = 1, \qquad \forall k \tag{6}$$

$$s_k \geq R_{kb} - p_b, \qquad \forall k, b \in \mathfrak{B} \tag{7}$$

$$p_b - R_{\max}(1 - \theta_{kb}) \leq P_{kb}, \qquad \forall k, b \in \mathfrak{B} \tag{8}$$

$$P_{kb} \leq p_b, \qquad \forall k, b \in \mathfrak{B} \tag{9}$$

$$s_k = \sum_{b \in \mathfrak{B}} (R_{kb}\theta_{kb} - P_{kb}), \qquad \forall k \tag{10}$$

$$R_{kb}\theta_{kb} - P_{kb} \geq 0, \qquad \forall k, b \in \mathfrak{B} \tag{11}$$

$$s_k \geq \sum_{b \in \mathfrak{B}} (R_{kb}\theta_{\gamma b} - P_{\gamma b}), \qquad \forall k, \gamma \tag{12}$$

$$Z_{kb} = P_{kb} - c_{kb}\theta_{kb}, \qquad \forall k, b \in \mathfrak{B} \tag{13}$$

$$p_b \leq p_{b_1} + p_{b_2}, \qquad \text{if } A(b) = A(b_1) \cup A(b_2) \tag{14}$$

$$p_b, P_{kb}, s_k \geq 0, s_{k,0} = 0, \theta_{kb} \in \{0, 1\} \tag{15}$$

We note that we use a simplified price sub-additivity constraints in (13). We refer to Appendix A.3 for the detailed discussions.

In the original formulation in Hanson and Martin [1990], all bundles combinations are included in the formulation, that is $\mathfrak{B} = \mathfrak{F} = 2^{\{1,\ldots,n\}}$. We call the corresponding problem with complete combinations HM($\mathfrak{F}$). Meanwhile, one can restrict to a subset of bundles $\mathfrak{B}$ and solve a restricted version of the above problem with only $b \in \mathfrak{B}$. We call the corresponding problem $HM(\mathfrak{B})$.

We note that the main computational challenge of the above formulation lies in the exponential number of variables corresponding to all the possible bundles. In the following, we demonstrate how GNN framework can be leveraged to help solve large-scale bundle pricing problem efficiently.

4 GNN-Based Strategies

In this section, we design a Graph Neural Network (GNN)-based strategy to solve the optimal bundle pricing problem. Specifically, we employ a Graph Neural Network (GNN) architecture characterized by an edge-centric update mechanism. Unlike standard Graph Convolutional Networks (GCNs) that primarily focus on learning node embeddings over static edge attributes, our framework executes a dual update process: it iteratively refines both node and edge embeddings. Our model predicts a segment-product probability matrix $\mathbb{P} \in \mathbb{R}^{m \times n}$, where $\mathbb{P}_{kj}$ is the predicted probability that customer segment k selects product j, derived directly from the final learned edge embeddings. These probabilities serve as a compact representation of heterogeneous preferences and provide the basis for pruning the exponentially large bundle space. We describe our strategies in detail in the following sections.

4.1 Motivation for GNN

We motivate our choice of architecture by analyzing the structural limitations of standard Multilayer Perceptrons (MLPs) in the context of bundle pricing, and explaining how Graph Neural Networks (GNNs) effectively address these challenges:

Limitations of MLPs. Standard feed-forward networks face two fundamental bottlenecks when applied to combinatorial problems of this nature:

(1) **Inability to Generalize Across Problem Sizes**: MLPs are constrained by fixed input and output dimensions. Consequently, instances with different numbers of products or customer segments typically require training separate models. While zero-padding can accommodate smaller instances, this approach is inefficient and fails to generalize to instances larger than the training configuration. In contrast, GNNs operate on graph topologies of arbitrary size using shared weights, allowing for seamless scalability.
(2) **Lack of Permutation Invariance**: MLPs treat inputs as flattened feature vectors and lack the inductive bias to capture the relational structure among agents. In our context, reordering product or customer indices should not alter the optimal solution (i.e., the problem is permutation invariant). However, because MLPs assign weights based on specific input positions, permuting the input order arbitrarily changes the output. GNNs naturally respect this symmetry by applying the same local operations across all nodes and edges, ensuring consistent predictions regardless of indexing.

Advantages of GNNs. In contrast, GNNs operate directly on graph-based representations, where nodes correspond to entities (products and customer segments) and edges encode their interactions (utilities).

(1) **Scalability through Weight Sharing:** A key feature of GNNs is the weight-sharing mechanism, where the same learnable weight matrices are applied across all nodes and edges. This decouples the number of parameters from the graph size, allowing a model trained on small-scale instances to generalize effectively to large-scale instances without retraining.
(2) **Context-Aware Representations:** Through message passing, GNNs explicitly model the bipartite interactions. Each node iteratively aggregates information from its neighbors, building a representation that reflects both its local attributes (e.g., costs) and its structural context (e.g., demand from connected segments). This enables the model to capture the latent patterns of optimal bundles in a manner that is both scalable and permutation invariant.

4.2 Graph Attributes

Figure 1 illustrates the graph attributes of the optimal bundle pricing problem, which serves as the input to our Graph Neural Network (GNN). The graph consists of two types of nodes: product nodes (□) and customer nodes (◯), as shown in Figure 1.

Let $\mathcal{V}$ and $\mathcal{E}$ denote the sets of nodes and edges, respectively. We define the raw features for the problem instance as follows:

- **Product Features:** Let $\mathbf{y}_j^P \in \mathbb{R}^4$ denote the initial feature vector for product j ($1 \le j \le n$). It is constructed as $\mathbf{y}_j^P = \left[c_j^u,\ \frac{1}{m}\sum_{k=1}^{m} u_{kj},\ 0,\ 0\right]$.
- **Customer Features:** Let $\mathbf{y}_k^S \in \mathbb{R}^4$ denote the initial feature vector for customer segment k ($1 \le k \le m$). It is constructed as $\mathbf{y}_k^S = \left[0,\ 0,\ \alpha_k,\ c_k^s\right]$.
- **Edge Features:** Each edge connecting product j and customer k is characterized by the scalar utility value u_{kj}.

4.3 GNN Structure

We implement our model following the **Graph Neural Network (GNN) framework** proposed by [Battaglia et al., 2018], utilizing the Generalized Graph Convolution [Li et al., 2020] for effective message aggregation. Specifically, we construct a Graph Neural Network consisting of Ω identical

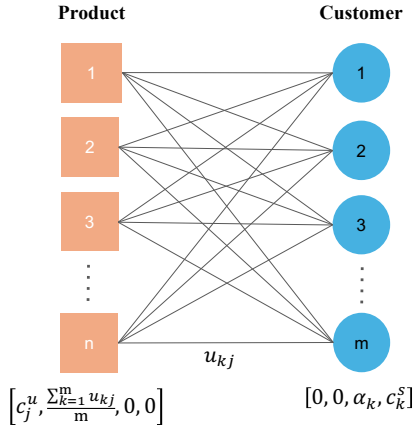

Fig. 1. Graph attributes of the bundle pricing problem with customer and product nodes.

Graph Network blocks. Each block performs relational reasoning by iteratively updating node and edge embeddings.

General Layer Notation. Let $G^{(l)} = (\mathbf{V}^{(l)}, \mathbf{Z}^{(l)})$ denote the graph embeddings at layer l.

The node feature matrix $\mathbf{V}^{(l)} \in \mathbb{R}^{(n+m)\times d_v^{(l)}}$ is defined as the vertical concatenation of the node attribute vectors. The matrix is constructed as:

$$\mathbf{V}^{(l)} = \begin{bmatrix} \mathbf{v}_1^{(l)} \\ \vdots \\ \mathbf{v}_{n+m}^{(l)} \end{bmatrix}. \tag{16}$$

Here, $\mathbf{v}_w^{(l)} \in \mathbb{R}^{1\times d_v^{(l)}}$ represents the embedding vector of the w-th node, and $d_v^{(l)}$ denotes the embedding dimensionality at layer l.

Furthermore, $\mathbf{V}^{(l)}$ can be explicitly partitioned into product and customer components. We denote $\mathbf{Y}^{P,(l)} \in \mathbb{R}^{n\times d_v^{(l)}}$ as the embedding matrix for the n product nodes and $\mathbf{Y}^{S,(l)} \in \mathbb{R}^{m\times d_v^{(l)}}$ as the embedding matrix for the m customer nodes. Thus:

$$\mathbf{V}^{(l)} = \begin{bmatrix} \mathbf{Y}^{P,(l)} \\ \mathbf{Y}^{S,(l)} \end{bmatrix}. \tag{17}$$

Similarly, the edge embedding tensor $\mathbf{Z}^{(l)} \in \mathbb{R}^{n\times m\times d_e^{(l)}}$ collects the embeddings for all pairs of products and customers. The relationship between the tensor $\mathbf{Z}^{(l)}$ and an individual edge embedding vector $\mathbf{e}_{jk}^{(l)} \in \mathbb{R}^{d_e^{(l)}}$ is given by:

$$\mathbf{Z}^{(l)}[j, k, :] = \mathbf{e}_{jk}^{(l)}, \quad \forall j \in \{1, \ldots, n\}, k \in \{1, \ldots, m\}, \tag{18}$$

where $d_e^{(l)}$ is the edge embedding dimensionality at layer l.

Initialization ($l = 0$). The input attributes for the GNN are initialized directly from the raw attribute vectors defined in Section 4.2. For node attributes, we map the product and customer

attributes to the global node index w:

$$\mathbf{v}_w^{(0)} = \begin{cases} \mathbf{y}_w^P & \text{if } 1 \le w \le n \quad \text{(Product Node)} \\ \mathbf{y}_{w-n}^S & \text{if } n < w \le n+m \quad \text{(Customer Node)} \end{cases} \tag{19}$$

For edge attributes, we initialize the tensor with the utility values:

$$\mathbf{e}_{jk}^{(0)} = [u_{kj}]. \tag{20}$$

Each GNN layer executes two update functions sequentially: a node update followed by an edge update. We provide a schematic illustration of the GNN architecture in Figure 2. Below, we describe the computations performed in each layer.

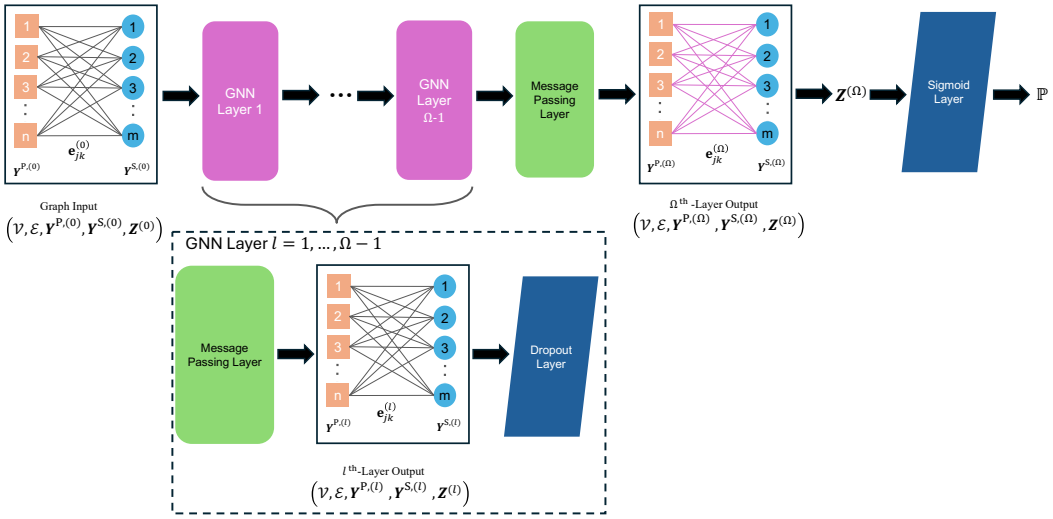

Fig. 2. Illustration of Graph Neural Network.

Node Update (ϕ^v). In the first stage of layer l, we update the embeddings of all nodes to capture the structural information from their neighbors. We employ the Generalized Graph Convolution as the update function. For a target node w and its neighbor set $\mathcal{N}(w)$, the update consists of three operations:

1. Message Construction: We construct a message vector $\mathbf{m}_{z\to w}^{(l)}$ for each neighbor $z \in \mathcal{N}(w)$. Since the dimensionality of edge embeddings may differ from that of node embeddings (e.g., in the input layer), we apply a learnable linear projection $\mathbf{W}_e^{(l)}$ to the edge embedding before summation. To ensure numerical stability and gradient flow, a ReLU activation and a small constant ϵ are applied:

$$\mathbf{m}_{z\to w}^{(l)} = \text{ReLU}\left(\mathbf{v}_z^{(l-1)} + \mathbf{W}_e^{(l)} \mathbf{e}_{wz}^{(l-1)}\right) + \epsilon \mathbf{1}. \tag{21}$$

Here, $\mathbf{e}_{wz}^{(l-1)}$ denotes the embedding of the edge connecting nodes w and z.

2. Softmax Aggregation: We aggregate these messages using a Softmax mechanism, which allows the model to learn adaptive weights for different neighbors:

$$\hat{\mathbf{m}}_w^{(l)} = \sum_{z\in\mathcal{N}(w)} \left(\frac{\exp(\beta \cdot \mathbf{m}_{z\to w}^{(l)})}{\sum_{\gamma\in\mathcal{N}(w)} \exp(\beta \cdot \mathbf{m}_{\gamma\to w}^{(l)})} \right) \odot \mathbf{m}_{z\to w}^{(l)}, \tag{22}$$

where β is a temperature parameter and $\odot$ denotes element-wise multiplication.

3. Residual Update: Finally, the aggregated message is combined with the node's own previous embedding and passed through an MLP:

$$\mathbf{v}_w^{(l)} = \mathrm{MLP}_{node}^{(l)}\left(\mathbf{v}_w^{(l-1)} + \hat{\mathbf{m}}_w^{(l)}\right). \tag{23}$$

Edge Update (ϕ^e). In the second stage, we explicitly update the embedding of each edge. Unlike standard GNNs where edges are static, our framework refines edge features $\mathbf{e}_{jk}^{(l)}$ based on the newly updated nodes and the edge's own history.

For an edge connecting product j and customer segment k (with global index $w = n + k$), we first construct a concatenated input vector $\mathbf{z}_{jk}^{(l)}$:

$$\mathbf{z}_{jk}^{(l)} = \mathbf{v}_j^{(l)} \,\|\, \mathbf{v}_{n+k}^{(l)} \,\|\, \mathbf{e}_{jk}^{(l-1)}, \tag{24}$$

where $\|$ denotes concatenation. This vector is then processed by a two-layer MLP with a residual-style logic and Layer Normalization:

$$\mathbf{e}_{jk}^{(l)} = \mathrm{LayerNorm}\left(\mathbf{W}_2 \cdot \mathrm{ReLU}(\mathbf{W}_1\mathbf{z}_{jk}^{(l)} + \mathbf{b}_1) + \mathbf{b}_2\right). \tag{25}$$

Here, $\{\mathbf{W_1}, \mathbf{W_2}\}$ and $\{\mathbf{b_1}, \mathbf{b_2}\}$ represent the learnable weights and biases of the edge update network.

Output. The model outputs the logits $\mathbf{Z}^{\Omega} \in \mathbb{R}^{n\times m}$. In practice, we apply a sigmoid function at inference time to obtain probabilities, $\mathbb{P} = \sigma(\mathbf{Z}^{\Omega})^{\top} \in \mathbb{R}^{m\times n}$, where each entry $\mathbb{P}_{kj}$ corresponds to the predicted probability that segment k include product j in his/her purchased bundle.

4.4 Training Process

To train the Graph Neural Network, we first collect historical data problem instances, and then use the Mixed Bundling method proposed by Hanson and Martin [1990] to obtain the corresponding optimal bundle assignments θ_{kb}^*'s, and they are converted to the ground truth labels for the GNN training. Specifically, our training procedure adopts a supervised learning paradigm, where the GNN is trained to predict selection probabilities for each customer–product pair, serving as a probabilistic predictor of the underlying binary product-selection decisions. Importantly, the model does not learn bundle prices directly; instead, it learns to approximate the optimal bundle assignment structure implied by the Mixed Bundling solution, which is leveraged to conduct high-quality search space pruning before solving the MILP, thereby accelerating the solution process by reducing computational complexity while maintaining near-optimal solution quality.

$$q_{kj}^* = \begin{cases} 1 & \text{if product } j \text{ is included in the optimal bundle for segment } k \\ 0 & \text{otherwise} \end{cases} \tag{26}$$

Thus, each instance ℓ is characterized by $(\mathbf{c}_{(\ell)}^u, \mathbf{c}_{(\ell)}^s, \mathbf{U}_{(\ell)}, \boldsymbol{\alpha}_{(\ell)}, f(\cdot), \mathbf{Q}_{(\ell)}^*)$, where $\mathbf{U}_{(\ell)} = [u_{kj,(\ell)}]_{m\times n}$ and $\mathbf{Q}_{(\ell)}^* = [q_{kj,(\ell)}^*]_{m\times n}$. The instances were split into a training set (80%) and a validation set (20%).

We then convert each problem instance into the corresponding graph representation and construct the input tensors $\mathbf{Y}^P$ and $\mathbf{Y}^S$. Then we formulate the edge prediction task as a binary classification problem. During each training epoch, we first pass the graphs in the training set to the Graph Neural Network to obtain the predicted product selection likelihoods $\mathbb{P}_{kj}$'s. We compute the weighted binary cross-entropy loss $\mathcal{L}$. Let S_T denote the number of instances in the training set. To address the inherent class imbalance (as optimal bundles typically contain only a small subset of available products), we introduce a positive weight parameter $w_{pos} = (1-\kappa)/\kappa$, where

$\kappa = \frac{\sum_{\ell=1}^{S_T} \sum_{k=1}^{m_{(\ell)}} \sum_{j=1}^{n_{(\ell)}} q^*_{kj,(\ell)}}{\sum_1^{S_T} m_{(\ell)} n_{(\ell)}}$ is the proportion of positive edges across the entire training set. The loss function is defined as:

$$\mathcal{L} = -\sum_{\ell=1}^{S_T} \sum_{k=1}^{m_{(\ell)}} \sum_{j=1}^{n_{(\ell)}} \left[w_{pos} \cdot q^*_{kj,(\ell)} \cdot \log(\mathbb{P}_{kj,(\ell)}) + (1 - q^*_{kj,(\ell)}) \cdot \log(1 - \mathbb{P}_{kj,(\ell)}) \right]. \tag{27}$$

Adam optimizer is used to update the model weights to minimize the training loss. Generalization performance is tracked by calculating the weighted binary cross-entropy loss on a validation set after every epoch. We employ a "best-model" retention strategy, replacing the stored model whenever a new epoch achieves a lower validation loss. While the Adam optimizer does not use the validation data for parameter updates, we leverage it for model selection to ensure the final network demonstrates strong generalization from the training set to validation set. Moreover, we employ an early-stopping strategy to mitigate overfitting and avoid redundant training iterations. Specifically, the training process terminates when the validation loss stagnates for a duration determined by the *patience* parameter. The resulting model is then applied to the inference policies described in the next subsection.

4.5 Pruning-based Inference Policies

In this subsection, we leverage the probability matrix $\mathbb{P}$ predicted by the GNN model to guide two pruning strategies that construct a compact yet high-quality bundle space. The resulting reduced problem is then solved using the MILP formulation of Hanson and Martin [1990], but restricted to the pruned bundle set rather than the full exponential space.

Fixed Cutoff Pruning (FCP). Our first policy generates one candidate bundle for each customer segment (Algorithm 2). For each segment $k \in [m] = \{1, \ldots, m\}$ and $j \in [n] = \{1, \ldots, n\}$, we define its candidate bundle by a fixed cutoff value. In our default method, we set the fixed cutoff to 0.5.[2] That is $B_k^{\text{FCP}} = \{j \in [n] \mid \mathbb{P}_{kj} \geq 0.5\}$ for $k \in [m]$ and the overall candidate set is $\mathfrak{F}^{\text{FCP}} = \{B_k^{\text{FCP}} \mid k \in [m]\}$, with size at most m. Here, the overall candidate set is shared across all segments, meaning that any segment is free to select any B_k^{FCP}, not only its own. This reduces the feasible space from 2^n to $O(m)$, making the subsequent MILP much more tractable. Then we solve the MILP formulation on the pruned space constructed by FCP (that is, we solve $HM(\mathfrak{F}^{\text{FCP}})$ to obtain the solution).

In the case where all probabilities of a segment fall below the cutoff, we retain the single product with the highest probability to avoid producing an empty bundle.

Progressive Cutoff Pruning (PCP). Our second policy generates a nested candidate bundle set for each customer segment (Algorithm 3). For each customer k, we first sort products by descending predicted probability and retain those with $\mathbb{P}_{kj} \geq 0.5$. Let $U_k = \left(j_{(1)}, j_{(2)}, \ldots, j_{(|U_k|)}\right)$ denote this ordered set of products.

We construct a sequence of prefix bundles, where $B_{k,i}^{\text{PCP}}$ denotes the i-th candidate bundle for customer k, consisting of the top i products, i.e., $B_{k,i}^{\text{PCP}} = \{j_{(1)}, j_{(2)}, \ldots, j_{(i)}\} \quad \text{for } i = 1, \ldots, |U_k|$. We then define $\mathcal{B}_k^{\text{PCP}}$ as the set of all candidate bundles specific to customer segment k, i.e., $\mathcal{B}_k^{\text{PCP}} = \{B_{k,i}^{\text{PCP}} \mid i = 1, \ldots, |U_k|\}$. Finally, the overall candidate set $\mathfrak{F}^{\text{PCP}}$ is defined as the union of these customer-wise sets, i.e., $\mathfrak{F}^{\text{PCP}} = \bigcup_{k=1}^{m} \mathcal{B}_k^{\text{PCP}}$.

This approach results in a pruned search space with a size of at most $mn + 1$. We then solve the restricted MILP formulation on this set, denoted as $HM(\mathfrak{F}^{\text{PCP}})$. Note that for any specific segment

[2]The choice of the cutoff is not tuned ex post. Appendix C provides a sensitivity analysis over a wide range of cutoff values, showing that 0.5 offers a robust revenue–runtime trade-off.

k, PCP produces a strictly nested chain of bundles:

$$B_{k,1}^{\text{PCP}} \subset B_{k,2}^{\text{PCP}} \subset \cdots \subset B_{k,|U_k|}^{\text{PCP}}.$$

Due to this nested structure, we replace the computationally expensive sub-additivity constraints within each segment with a simple chain of monotone price inequalities:

$$p(B_{k,i}^{\text{PCP}}) \leq p(B_{k,i+1}^{\text{PCP}}) \quad \text{for } i = 1, \ldots, |U_k| - 1.$$

Standard sub-additivity constraints are maintained for bundles across different customer segments.

Figure 3 illustrates the construction of candidate bundles under the FCP and PCP policies for a scenario with $m_{test} = 1$ and $n_{test} = 5$. First, products 2 and 4 are excluded from consideration as their predicted probabilities fall below the cutoff of 0.5. Under the FCP policy, the remaining high-probability products form a single candidate bundle, i.e., $B_1^{\text{FCP}} = \{1, 3, 5\}$. In contrast, under the PCP policy, given the probability order $\mathbb{P}_{11} > \mathbb{P}_{15} > \mathbb{P}_{13}$, the products are added sequentially to form a nested chain. Consequently, the candidate bundle set for customer 1 is given by $\mathcal{B}_1^{\text{PCP}} = \big\{\{1\}, \{1, 5\}, \{1, 5, 3\}\big\}$.

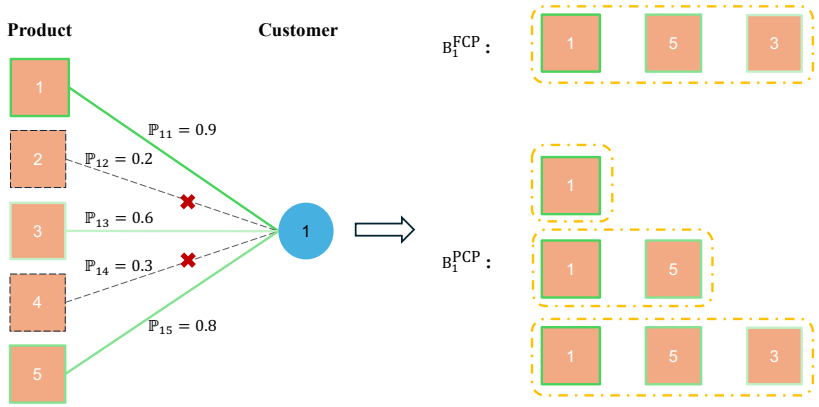

Fig. 3. Candidate Bundle Constructions under FCP and PCP policies

Pruning Effect on Model Size. We analyze the theoretical complexity of our pruning strategies to evaluate how they reduce the effective optimization problem size relative to the full MILP formulation. The full MILP model suffers from the curse of dimensionality, with a bundle space of size $2^n - 1$, leading to variables and constraints that scale exponentially as $O(m \cdot 2^n)$.

FCP performs the most aggressive space pruning by retaining at most one candidate bundle per segment. This strategy collapses the bundle space from exponential 2^n to linear $O(m)$, effectively transforming the problem complexity from exponential to quadratic $O(m^2)$ in terms of decision variables and constraints.

PCP adopts a more conservative pruning strategy by retaining a progressive chain of subsets for each segment, resulting in a bundle space size of $O(mn)$. Consequently, the number of decision variables and constraints scales as $O(m^2 n)$. While this complexity is slightly higher than FCP, it remains polynomial and avoids the exponential explosion of the full MILP.

Overall, both FCP and PCP achieve highly effective pruning, substantially reducing the optimization problem size and improving model construction efficiency. This advantage is expected to become even more pronounced as the number of products n increases, where the full bundle space grows exponentially. Importantly, thanks to the high-quality bundle probability predictions

provided by our GCN model, both strategies are able to maintain near-optimal solution quality despite operating on a heavily pruned bundle space, which will be demonstrated in the numerical results.

Table 4.1. Theoretical Complexity Comparison of MILP, FCP, and PCP Strategies

Strategy	Bundle Space ($\|\mathcal{B}\|$)	Variables ($O(m \cdot \|\mathcal{B}\|)$)	Total Constraints ($O(m \cdot \|\mathcal{B}\| + m^2)$)
MILP	2^n	$O(m \cdot 2^n)$	$O(m \cdot 2^n)$
FCP	$O(m)$	$O(m^2)$	$O(m^2)$
PCP	$O(mn)$	$O(m^2 n)$	$O(m^2 n)$

Note: The variable and constraint counts are derived from the mixed integer programming formulation used in the bundle pricing problem.

4.6 GNN Guided Local Search Policy

In order to further improve our solution, we propose an inference policy based on GNN-guided local search. The basic idea is to iteratively modify the bundle assignment by either adding an unselected product or dropping a selected product, and accept modifications if the profit increases.

To maximize the efficiency of the local search, we develop a **Global Top-K Local Search strategy** guided by the probability matrix $\mathbb{P}$ predicted by the GNN model. Instead of evaluating neighbors for all segments, we focus only on the most promising modifications globally and prioritize candidates based on a confidence score.

Specifically, for any segment k and product j, we define the confidence score for a potential modification as:

$$\text{Score}_{kj} = \begin{cases} \mathbb{P}_{kj} & \text{if product } j \text{ is currently unselected (Candidate for Addition)} \\ 1 - \mathbb{P}_{kj} & \text{if product } j \text{ is currently selected (Candidate for Removal)} \end{cases} \tag{28}$$

Intuitively, a higher $\mathbb{P}_{kj}$ indicates a strong suggestion to include the product, while a lower $\mathbb{P}_{kj}$ implies a strong suggestion to exclude it.

In each iteration, we independently identify the top K 'Add' candidates and top K 'Drop' candidates based on their confidence scores. These candidates are then pooled together and re-sorted by score for sequential evaluation. We define the candidate size as $K = \lceil \sqrt{m} \rceil$. This sublinear scaling strikes a critical balance: it avoids the linear computational growth associated with exhaustive segment-wise search (where candidates scale with m), while ensuring that the neighborhood size expands sufficiently to maintain solution stability as the problem dimension increases—a robustness often lacking in constant-K heuristics. As detailed in Appendix D, this strategy achieves the highest computational efficiency ratio while preserving near-optimal revenue performance. These neighbors are then evaluated sequentially in descending order of their scores. If any improvement is found, the modification is accepted immediately (greedy strategy), and a new iteration begins.

The local search process terminates when no improvement is found among the top candidates in a full cycle, or when the predefined iteration limit is reached. Furthermore, we rely on the LP relaxation to efficiently evaluate each neighbor, as it provides a valid lower bound for the MILP objective and significantly reduces computational cost. We demonstrate a high degree of consistency between LP and MILP objective improvements during the local search process, justifying the use of LP as a reliable proxy in Appendix E. We use FCPLS to denote this algorithm.

5 Numerical Experiments

We now present comprehensive numerical experiments designed to evaluate the effectiveness and scalability of our proposed GNN-assisted bundle pricing strategies. The goals of this section are twofold: (i) to demonstrate that our approaches consistently achieve high competitive ratios while maintaining significant computational efficiency, and (ii) to validate their robustness across heterogeneous problem settings with varying numbers of customer segments and products.

To thoroughly test scalability and robustness, we evaluate our strategies under varying numbers of customer segment m and product number n. For each scenario, results are averaged over 100 instances. The GNN model training was conducted with a single NVIDIA A100 GPU, while the GNN model inference and the inference policies were performed on a local machine featuring an Apple M1 Pro CPU (3.2 GHz) and 16GB RAM, utilizing up to 8 threads. The implementation uses Gurobi 12.0, Python 3.10, PyTorch 2.8, and CUDA 12.8.

To evaluate any method, we adopt two normalized metrics: *Competitive Ratio (CR)* and *Time Ratio (TR)*

$$CR_{A,B} = \frac{\text{Profit of approach A}}{\text{Profit of approach B}}, \quad TR_{A,B} = \frac{\text{Runtime of approach A}}{\text{Runtime of approach B}}.$$

where CR measures solution quality and TR captures efficiency. These two metrics allow direct comparison of solution quality and speed across different algorithms.

5.1 Basic Simulations

We compare our proposed approach against the two most fundamental and widely recognized policies in the literature: the exact Mixed Bundling (MB) policy and the approximation-based Bundle Size Pricing (BSP) policy:

(1) **Mixed Bundling (MB) policy.** For MB, we follow Hanson and Martin [1990]'s MILP formulation. The MB formulation is a MILP that assigns each bundle with its own price respectively. Detailed formulation is provided in Section 3.1. We solve the MB policy with a MIPGap threshold of 0.1%.
(2) **Bundle Size Pricing (BSP) policy.** The BSP baseline is proposed by Chu et al. [2011]. The BSP approximates MB by assigning the same price to all bundles of equal size. Detailed formulation is provided in Appendix A.1. We solve the BSP policy with an MIPGap threshold of 0.1%.

Remark 2 (Limitation of Other Policies). *It is worth noting that we exclude certain heuristic algorithms and general Neural MILP methods from our baselines due to their inapplicability to the general bundle pricing problem.*

First, existing approximation algorithms such as CPBSD [Chen et al., 2022] and PBDC [Ma and Simchi-Levi, 2021] are designed primarily under the strict assumption of additive valuations. Their formulations express the valuation of a bundle as a product-wise summation of individual item values. However, in a subadditive setting where substitution and complementarity effects (sub-additivity and super-additivity) are present, the valuation of a bundle is not equal to the simple sum of its constituent products. As a result, these additive formulations cannot be directly applied to the more complex non-additive bundle valuation structure considered in our setting.

Second, recent neural-network-based MILP approaches, such as Neural Branching [Gasse et al., 2019] or Neural Diving [Nair et al., 2020] algorithms, are computationally infeasible for this problem. These methods typically rely on encoding the MILP into a variable-constraint bipartite graph. However, since the standard formulation of mixed bundling involves an exponential number of decision variables (2^n products), constructing and processing such a graph is almost impossible.

Solution Sample Generation. Let $\mathcal{U}[a, b]$ denote the uniform distribution over the interval $[a, b]$, and let $\mathcal{U}_D[N]$ denote the discrete uniform distribution over the set $[n]$. In the numerical experiments, given the number of products n, the number of customer types M, we generate each problem instance as follows:

- **Customer Proportions.** We first sample m value β_k from $\mathcal{U}[0, 1]$ and normalize to obtain the customer proportions $\boldsymbol{\alpha} = (\alpha_1, \cdots, \alpha_k)$, where $\alpha_k = \beta_k / \sum_{k'=1}^{m} \beta_{k'}$.
- **Product Utilities and Costs.** For each product $j \in [n]$ and customer segment $k \in [m]$, we draw utilities u_{kj} from $\mathcal{U}[0, 1]$, unit costs c_j^u from $\mathcal{U}[0, 0.2]$ and ship costs c_k^s from $\mathcal{U}[0, 0.2]$. Then, the bundle costs and valuations can be computed by $c_{kb} = \left(\sum_{j \in b} c_j^u\right) + c_k^s$ and $R_{kb} = \sqrt{\sum_{j \in b} u_{kj}}$.
- **Label Generation.** Finally, for each generated instance, we compute the bundle assignment θ_{kb}^*'s using MB policy and convert them into the ground truth labels q_{kj}^*'s. The resulting sample is characterized by $(\mathbf{c}^u, \mathbf{c}^s, \mathbf{U}, \boldsymbol{\alpha}, \sqrt{\cdot}, \mathbf{Q}^*)$.

We employ a GNN model trained on 3000 samples with dimensions $m_{train} = 10$ customer types and $n_{train} = 10$ products. The ground truth labels for these training samples are obtained by solving the exact MB policy to optimality.

The dataset is then randomly divided into a training set (80%) and a validation set (20%). We use hidden dimension $d_{hid} = 128$ and $\Omega = 4$ GNN layers. The model is optimized using the Adam optimizer [Kingma, 2014] with a learning rate of 0.01 and a batch size of 128. To mitigate the impact of random initialization, we trained 10 separate GNN models using seeds 1 to 10 and reported the average performance of inference policies across these 10 models. Training is capped at 200 epochs per trial, incorporating an early-stopping criterion (patience=50) that halts training if the reduction in validation loss remains less than 10^{-12} for 50 consecutive epochs. We set an MIPGap threshold of 0.1% when solving MILPs in FCP, PCP and FCPLS policies.

In Table 5.1, we report the numerical results of our algorithms compared with both baselines, for problem with $n = 10$ and varying number of segments. From Table 5.1, we can see that our three strategies all maintain more than 96% of the optimal profit while only requiring a fraction of the computation time of the MB baseline. Meanwhile, the BSP approach often has a significant profit loss compared to the MB baseline. Among the strategies we propose, we find that the PCP strategy achieves over 1% higher profit than FCP by retaining a larger candidate space, while FCP+LS also gives a 1% profit improvement over the plain FCP approach.

Table 5.1. Performance of GNN-based inference policies compared with the MB policy ($n = 10$).

Strategy	$m_{test} = 10, n_{test} = 10$			$m_{test} = 20, n_{test} = 10$			$m_{test} = 30, n_{test} = 10$		
	$CR._{,MB}$ (std)	Time (s)	$TR._{,MB}$	$CR._{,MB}$ (std)	Time (s)	$TR._{,MB}$	$CR._{,MB}$ (std)	Time (s)	$TR._{,MB}$
FCP	0.973 (0.019)	0.047	0.007	0.968 (0.014)	0.308	0.010	0.963 (0.025)	1.043	0.006
PCP	0.985 (0.010)	0.252	0.038	0.985 (0.007)	2.978	0.070	0.985 (0.009)	20.436	0.076
FCPLS	0.985 (0.018)	0.252	0.038	0.978 (0.012)	1.754	0.056	0.971 (0.026)	6.293	0.037
BSP	0.869 (0.036)	0.051	0.008	0.858 (0.028)	0.262	0.008	0.858 (0.027)	0.753	0.004

For problems with more than 10 products, calculating the optimal solution of the MILP model under the MB policy is computationally challenging. Therefore, we use the BSP as baseline to test problems with larger than 10 products. The results are reported in Table 5.2. From Table 5.2, we can see that the plain FCP approach can achieve significantly better performance compared with the

BSP approach with a fraction of computation time, while the PCP approach achieves even higher optimality with more computational time (the total time is still less than 50 seconds).

Table 5.2. Comparison of different inference policies across various problem sizes compared with the BSP policy.

	FCP			PCP			FCPLS		
Scenario	$CR._{,BSP}$ (std)	Time (s)	TR	$CR._{,BSP}$ (std)	Time (s)	TR	$CR._{,BSP}$ (std)	Time (s)	TR
$m_{test} = 10, n_{test} = 20$	1.133 (0.036)	0.044	0.288	1.160 (0.039)	1.109	7.063	1.150 (0.037)	0.294	1.907
$m_{test} = 10, n_{test} = 40$	1.076 (0.042)	0.045	0.103	1.191 (0.037)	7.947	17.331	1.113 (0.038)	0.420	0.960
$m_{test} = 20, n_{test} = 20$	1.150 (0.032)	0.262	0.226	1.181 (0.034)	17.714	14.334	1.165 (0.033)	1.918	1.677
$m_{test} = 20, n_{test} = 40$	1.097 (0.037)	0.267	0.066	1.212 (0.035)	48.842	11.782	1.125 (0.035)	2.710	0.663

Scalability across the number of customer segments m_{test}. Under a strict 600-second time limit, Figure 4 illustrates the scalability of the FCP and PCP policies as the number of customer segments m_{test} varies ($n_{test} = 20$). The FCP policy demonstrates robust scalability, consistently outperforming the BSP policy while requiring less than 30% of the computational time. In contrast, while the PCP policy yields higher profits for smaller instances ($m_{test} \leq 50$), it becomes computationally intractable for larger m_{test} within the time limit.

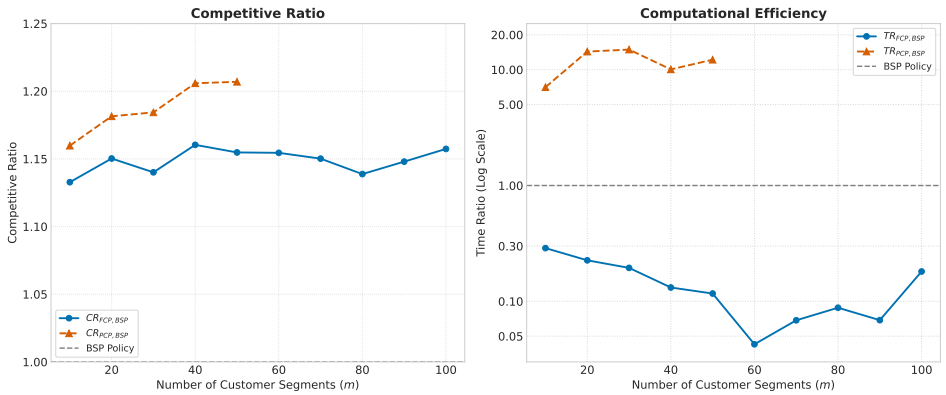

Fig. 4. Comparison of model scalability across m_{test} under different inference policies.

Scalability across the number of products n_{test}. Under a 600-second time limit, Figure 5 illustrates the scalability of the FCP and PCP policies as the number of products n_{test} varies ($m_{test} = 10$). The PCP policy demonstrates robust scalability, consistently achieving a competitive ratio exceeding 115% relative to the BSP policy. Conversely, while the FCP policy maintains superior computational efficiency, its performance degrades as the product size expands, yielding lower profits than the BSP policy when $n_{test} \geq 60$. To address this limitation, we propose an Iterative Self-Improvement strategy in the subsequent subsection.

5.2 Iterative Self-Improvement

From Figure 5, we observe that the model trained on small-scale data ($n = 10$) exhibits limited zero-shot generalization capability when applied to significantly larger product spaces, suggesting

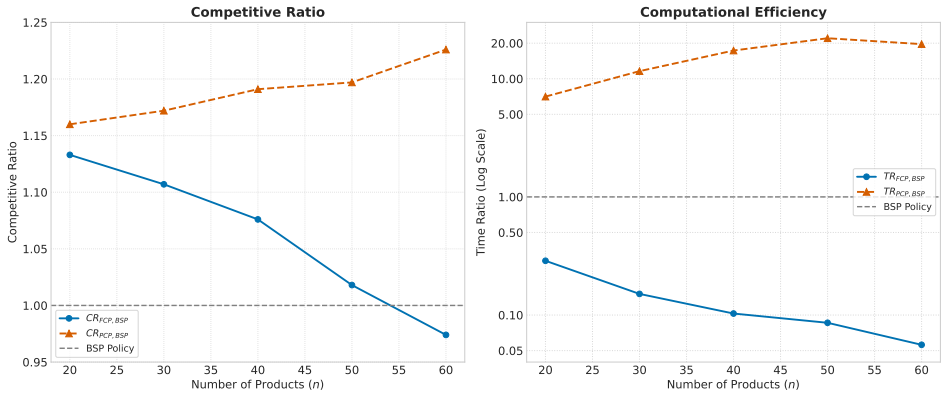

Fig. 5. Comparison of model scalability across n_{test} under different inference policies.

the necessity of training on larger instances. However, calculating the exact optimal solutions for such large-scale problems via the MB policy is impossible due to the combinatorial explosion. To resolve this dilemma, we propose an **Iterative Self-Improvement strategy** to progressively scale our framework. We utilize our proposed framework to bootstrap the learning process. Specifically, we first train a base model on small-scale instances (e.g., $m_{train} = 10, n_{train} = 10$) where optimal solutions are easily obtainable. We then deploy this base model with the PCP policy to generate high-quality—albeit suboptimal—solutions for larger-scale instances. Since PCP policy achieves superior optimality as shown in Table 5.1 and Table 5.2, these solutions preserve the key features of the true optima (similar patterns are observed in Li et al. [2025]). These PCP-generated solutions serve as ground truth labels to train a new, more capable model. This cycle allows us to continuously expand the training scale and refine the model's performance on larger problems without hitting the computational bottleneck of MB policy.

In the numerical experiments, we first train a GNN model using samples with dimension $m_{train} = 10, n_{train} = 10$ following the setting in Subsection 5.1. Then, we generate 3000 suboptimal samples with dimension $m_{train} = 10$ and $n_{train} = 50$ calculated by PCP policy, and we train a separate GNN model using these 3000 samples with $m_{train} = 10$ and $n_{train} = 50$.

Table 5.3 demonstrates the efficacy of the Iterative Self-Improvement strategy. Specifically, it yields a substantial enhancement for the FCP policy, increasing the competitive ratio (relative to BSP) by approximately 20%. For the PCP policy, while the improvement in competitive ratio is a modest 2%, the computational cost is drastically reduced. These results underscore that the quality of the GNN model's predictions is pivotal, significantly influencing both the optimality of the final solution and the computational efficiency. By iteratively refining these predictions, the proposed strategy substantially enhances the model's scalability across number of products.

Table 5.3. Comparison of model performance under different training configurations.

	FCP ($m_{train} = 10, n_{train} = 10$)		**FCP** ($m_{train} = 10, n_{train} = 50$)		**PCP** ($m_{train} = 10, n_{train} = 10$)		**PCP** ($m_{train} = 10, n_{train} = 50$)	
Scenario	$CR_{\cdot,BSP}$ (std)	Time (s)	$CR_{\cdot,BSP}$ (std)	Time (s)	$CR_{\cdot,BSP}$ (std)	Time (s)	$CR_{\cdot,BSP}$ (std)	Time (s)
$m_{test} = 10, n_{test} = 50$	1.018 (0.046)	0.055	**1.190 (0.047)**	**0.047**	1.197 (0.039)	14.321	**1.212 (0.045)**	**1.595**
$m_{test} = 10, n_{test} = 60$	0.974 (0.054)	0.050	**1.211 (0.051)**	**0.046**	1.226 (0.047)	17.996	**1.246 (0.049)**	**2.776**

Table 5.4. Comparison of different inference policies across various problem sizes trained on suboptimal samples of size $m_{train} = 10, n_{train} = 50$.

	FCP			PCP			FCPLS		
Scenario	$CR._{,BSP}$ (std)	Time (s)	TR	$CR._{,BSP}$ (std)	Time (s)	TR	$CR._{,BSP}$ (std)	Time (s)	TR
$m_{test} = 10, n_{test} = 60$	1.211 (0.050)	0.046	0.050	1.246 (0.049)	2.776	3.028	1.225 (0.051)	0.268	0.295
$m_{test} = 10, n_{test} = 80$	1.186 (0.056)	0.050	0.038	1.260 (0.048)	7.006	5.363	1.204 (0.055)	0.304	0.240
$m_{test} = 10, n_{test} = 100$	1.167 (0.068)	0.051	0.029	1.283 (0.060)	8.316	4.814	1.189 (0.067)	0.344	0.200
$m_{test} = 20, n_{test} = 60$	1.229 (0.045)	0.223	0.034	1.265 (0.043)	20.520	3.113	1.238 (0.045)	1.521	0.233
$m_{test} = 20, n_{test} = 80$	1.222 (0.055)	0.241	0.027	1.289 (0.047)	32.309	3.573	1.232 (0.054)	1.684	0.191
$m_{test} = 20, n_{test} = 100$	1.193 (0.053)	0.254	0.022	1.304 (0.045)	47.680	4.011	1.207 (0.052)	2.042	0.174

Algorithm 1 Iterative Self-Improvement Strategy

Input: Small-scale dimensions (m_{small}, n_{small}); Large-scale dimensions (m_{large}, n_{large}); Exact Solver π_{MB}; Inference Policy π_{PCP}.
Output: Trained GNN model ζ_{new} for large-scale instances.

Phase 1: Base Model Training (Small Scale)
Generate N_{small} historical instances $\mathcal{I}_{small}$ with dimensions (m_{small}, n_{small}).
Initialize dataset $\mathcal{D}_{base} \leftarrow \emptyset$.
for each instance $\ell \in \mathcal{I}_{small}$ **do**
  Obtain optimal bundle assignment $\mathbf{Q}^*_{(\ell)} \leftarrow \pi_{\text{MB}}(\text{instance } \ell)$. ▷ Exact labels via MB
  $\mathcal{D}_{base} \leftarrow \mathcal{D}_{base} \cup \{(\text{instance } \ell, \mathbf{Q}^*_{(\ell)})\}$.
end for
Split $\mathcal{D}_{base}$ into training/validation sets (80%/20%).
Initialize GNN parameters ζ_{base}.
$\zeta_{base} \leftarrow \text{TrainGNN}(\zeta_{base}, \mathcal{D}_{base})$. ▷ Train base model on exact labels

Phase 2: Self-Improvement via Bootstrapping (Large Scale)
Generate N_{large} instances $\mathcal{I}_{large}$ with dimensions (m_{large}, n_{large}).
Initialize dataset $\mathcal{D}_{new} \leftarrow \emptyset$.
for each instance $\ell' \in \mathcal{I}_{large}$ **do**
  Predict selection probabilities $\mathbb{P}_{(\ell')} \leftarrow \text{GNN}(\text{instance } \ell'; \zeta_{base})$.
  Generate solution $\hat{\mathbf{Q}}^*_{(\ell')} \leftarrow \pi_{\text{PCP}}(\mathbb{P}_{(\ell')})$. ▷ Suboptimal labels via PCP
  $\mathcal{D}_{new} \leftarrow \mathcal{D}_{new} \cup \{(\text{instance } \ell', \hat{\mathbf{Q}}^*_{(\ell')})\}$.
end for
Split $\mathcal{D}_{new}$ into training/validation sets (80%/20%).
Initialize GNN parameters ζ_{new}.
$\zeta_{new} \leftarrow \text{TrainGNN}(\zeta_{new}, \mathcal{D}_{new})$. ▷ Train new model on pseudo-labels
return ζ_{new}

6 Conclusion

This paper introduces a learning-guided framework for solving the bundle pricing problem. We leverage GNNs to learn segment–product preference structures under the non-additive setting, and use these predictions to prune the exponential candidate bundle space. The pruned feasible region is then solved with Hanson and Martin [1990]'s MILP formulation, thereby retaining the

rigor of product heterogeneity while substantially extending the tractable problem size. Coupled with a probability-guided local search, our framework demonstrates robustness and scalability across different problem sizes. Numerical experiments show that our approach provides solutions that are near-optimal at tractable scales, and scalable to much larger settings while outperforming other heuristics. Therefore, our work provides a near-optimal and efficient solution for large-scale bundle pricing problem.

Finally, while our current framework relies on deterministic valuations, incorporating probabilistic consumer behavior represents a significant avenue for future research. We plan to extend our GNN-based approach to accommodate parametric/semiparametric choice models. By modeling the stochastic nature of consumer decisions where selection probabilities are driven by random valuations, we aim to validate the robustness of our pricing strategies and further bridge the gap between algorithmic pricing and real-world behavioral heterogeneity.

References

Tarek Abdallah. 2019. On the Benefit (Or Cost) of Large-Scale Bundling. *Production and Operations Management* 28, 4 (2019), 955–969.

Tarek Abdallah, Amir Asadpour, and Jared Reed. 2021. Large-Scale Bundle-Size Pricing: A Theoretical Analysis. *Operations Research* 69, 4 (2021), 1158–1185.

William James Adams and Janet L. Yellen. 1976. Commodity Bundling and the Burden of Monopoly. *The Quarterly Journal of Economics* 90, 3 (1976), 475–498.

Yannis Bakos and Erik Brynjolfsson. 1999. Bundling Information Goods: Pricing, Profits, and Efficiency. *Management Science* 45, 12 (1999), 1613–1630.

Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018).

Moran Beladev, Lior Rokach, and Bracha Shapira. 2016. Recommender systems for product bundling. *Knowledge-Based Systems* 111 (2016), 193–206.

Jianxin Chang, Chen Gao, Xiangnan He, Depeng Jin, and Yong Li. 2021. Bundle recommendation and generation with graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* 35, 3 (2021), 2326–2340.

Liang Chen, Yang Liu, Xiangnan He, Lianli Gao, and Zibin Zheng. 2019. Matching user with item set: Collaborative bundle recommendation with deep attention network.. In *IJCAI*. 2095–2101.

Ningyuan Chen, Adam N. Elmachtoub, Michael L. Hamilton, and Xiao Lei. 2021. Loot Box Pricing and Design. *Management Science* 67, 8 (2021), 4809–4825.

Ningyuan Chen, Xiaobo Li, Zechao Li, and Chun Wang. 2022. Component Pricing with a Bundle Size Discount. (2022). Available at SSRN: https://ssrn.com/abstract=4032247.

Chenghuan Sean Chu, Phillip Leslie, and Alan Sorensen. 2011. Bundle-Size Pricing as an Approximation to Mixed Bundling. *American Economic Review* 101, 1 (2011), 263–303.

Jian-Ya Ding, Chao Zhang, Lei Shen, Shengyin Li, Bing Wang, Yinghui Xu, and Le Song. 2020. Accelerating Primal Solution Findings for Mixed Integer Programs Based on Solution Prediction. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*. 1452–1459.

Zhenan Fan, Xinglu Wang, Oleksandr Yakovenko, Abdullah Ali Sivas, Owen Ren, Yong Zhang, and Zirui Zhou. 2023. Smart Initial Basis Selection for Linear Programs. In *Proceedings of the 40th International Conference on Machine Learning*. 9650–9664.

Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. 2019. Exact Combinatorial Optimization with Graph Convolutional Neural Networks. In *Proceedings of the 33rd Conference on Neural Information Processing Systems*. 15580–15592.

Grand View Research. 2023. Digital Media Market Size, Share & Growth Report. https://www.grandviewresearch.com/industry-analysis/digital-media-market-report

Qing Guo, Saman Lagzi, Chenhao Wang, Ningyuan Chen, Guillermo Gallego, Sumit Kunnumkal, Yao Wang, and Li Yu. 2025. Solving Assortment Optimization with First-Order Methods and Neural Networks: A Computational Framework and Public Benchmark. *Available at SSRN 5671592* (2025).

Prateek Gupta, Elias B. Khalil, Didier Chételat, Maxime Gasse, Yoshua Bengio, Andrea Lodi, and M. Pawan Kumar. 2022. Lookback for Learning to Branch. *arXiv preprint arXiv:2206.14987* (2022).

Ward Hanson and R. Kipp Martin. 1990. Optimal Bundle Pricing. *Management Science* 36, 2 (1990), 155–174.

Lorin M. Hitt and Pei-yu Chen. 2005. Bundling with Customer Self-Selection: A Simple Approach to Bundling Low-Marginal-Cost Goods. *Management Science* 51, 10 (2005), 1481–1493.

Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

Wouter Kool, Herke van Hoof, and Max Welling. 2019. Attention, Learn to Solve Routing Problems! *arXiv preprint arXiv:1803.08475* (2019).

Guokai Li, Pin Gao, Stefanus Jasin, and Zizhuo Wang. 2025. From Small to Large: A Graph Convolutional Network Approach for Solving Assortment Optimization Problems. *arXiv preprint arXiv:2507.10834* (2025).

Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. 2020. DeeperGCN: All you need to train deeper GCNs. *arXiv preprint arXiv:2006.07739* (2020).

Jiuqiang Li. 2025. Disentangled Contrastive Bundle Recommendation with Conditional Diffusion. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12067–12075.

Xiaobo Li, Hailong Sun, and Chung-Piaw Teo. 2021. Convex Optimization for the Bundle Size Pricing Problem. *Management Science* 68, 2 (2021), 1095–1106.

Tianhao Liu, Shanwen Pu, Dongdong Ge, and Yinyu Ye. 2024. Learning to Pivot as a Smart Expert. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence*. 8073–8081.

Will Ma and David Simchi-Levi. 2021. Reaping the benefits of bundling under high production costs. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 1342–1350.

Yunshan Ma, Yingzhi He, An Zhang, Xiang Wang, and Tat-Seng Chua. 2022. CrossCBR: cross-view contrastive learning for bundle recommendation. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 1233–1241.

Modern Retail. 2024. Inside Amazon's Prime playbook: How the subscription giant chooses what benefits to add next. https://www.modernretail.co/operations/inside-amazons-prime-playbook-how-the-subscription-giant-chooses-what-benefits-to-add-next/

Vinod Nair, Sergey Bartunov, Felix Gimeno, Ingrid von Glehn, Pawel Lichocki, Ivan Lobov, Brendan O'Donoghue, Nicolas Sonnerat, Christian Tjandraatmadja, Pengming Wang, Ravichandra Addanki, Tharindi Hapuarachchi, Thomas Keck, James Keeling, Pushmeet Kohli, Ira Ktena, Yujia Li, Oriol Vinyals, and Yori Zwols. 2020. Solving Mixed Integer Programs Using Neural Networks. *arXiv preprint arXiv:2012.13349* (2020).

Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder, and Martin Takác. 2018. Reinforcement learning for solving the vehicle routing problem. *Advances in neural information processing systems* 31 (2018).

Max Paulus and Andreas Krause. 2023. Learning to dive in branch and bound. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 34260–34277.

Oren Sar Shalom, Noam Koenigstein, Ulrich Paquet, and Hastagiri P Vanchinathan. 2016. Beyond collaborative filtering: The list recommendation problem. In *Proceedings of the 25th international conference on world wide web*. 63–72.

Richard Schmalensee. 1984. Gaussian Demand and Commodity Bundling. *The Journal of Business* 57, 1 (1984), 211–230.

Nicolas Sonnerat, Pengming Wang, Ira Ktena, Sergey Bartunov, and Vinod Nair. 2021. Learning a large neighborhood search algorithm for mixed integer programs. *arXiv preprint arXiv:2107.10201* (2021).

George J. Stigler. 1963. United States v. Loew's Inc.: A Note on Block-Booking. *The Supreme Court Review* (1963), 152–157.

A Formulations for Bundle Pricing

A.1 Bundle Size Pricing (BSP) Formulation Chu et al. [2011]

Additional Variables:

- p_s: price for bundles of size $s \in \{0, \ldots, n\}$;
- $P_{ks} = p_s \theta_{ks}$.

Objective:

$$\max \quad \sum_{k=1}^{m} \sum_{s=0}^{n} \alpha_k \cdot Z_{ks} \tag{29}$$

Constraints:

$$s_k \geq R_{ks} - p_s, \quad \forall k, s \tag{30}$$

$$\sum_{s=0}^{n} \theta_{ks} = 1, \quad \forall k \tag{31}$$

$$P_{ks} \leq p_s, \quad \forall k, s \tag{32}$$

$$P_{ks} \geq p_s - M(1-\theta_{ks}), \quad \forall k, s \tag{33}$$
$$Z_{ks} = P_{ks} - c_{ks}\theta_{ks}, \quad \forall k, s \tag{34}$$
$$s_k = \sum_{s=0}^{n} (R_{ks}\theta_{ks} - P_{ks}), \quad \forall k \tag{35}$$
$$s_k \geq \sum_{s=0}^{n} (R_{ks}\theta_{js} - P_{js}), \quad \forall k, j \neq k \tag{36}$$
$$R_{ks}\theta_{ks} - P_{ks} \geq 0, \quad \forall k, s \tag{37}$$
$$p_{s_1+s_2} \leq p_{s_1} + p_{s_2}, \quad \forall s_1, s_2 \text{ s.t. } s_1 + s_2 \leq n \tag{38}$$
$$p_{s+1} \geq p_s, \quad \forall s = 0, \ldots, n-1 \tag{39}$$
$$p_s, P_{ks}, Z_{ks} \geq 0, \quad \theta_{ks} \in \{0,1\} \quad \forall k, s$$

A.2 LP Formulation (Fixed Assignment)

Decision variables:

- $p_i \geq 0$: price of bundle $i \in F$, where F is the pruned candidate bundle set;
- s_k: surplus of segment $k \in \{1, \ldots, m\}$.

Objective:

$$\max_{p,s} \quad \sum_{k=1}^{m} \alpha_k \cdot \left(p_{b_k} - c_{k,b_k}\right), \tag{L.1}$$

where $b_k \in F$ is the fixed bundle assigned to segment k.

Constraints:

$$s_k \geq R_{k,i} - p_i \quad \forall k = 1, \ldots, m, \ \forall i \in F \qquad \text{(lower bound)} \tag{L.2}$$
$$s_k \leq R_{k,b_k} - p_{b_k} \quad \forall k \qquad \text{(assignment binding)} \tag{L.3}$$
$$p_i \leq p_{j_1} + p_{j_2}, \quad \text{if } A_i = A_{j_1} \cup A_{j_2}, \ \{j_1, j_2\} \subset F \qquad \text{(sub-additivity)} \tag{L.4}$$
$$p_0 = 0 \qquad \text{(empty bundle price)} \tag{L.5}$$

Remarks.

- Bundle assignments are fixed externally by assigning each segment with their exact FCP optimal bundle prediction: $\theta_{kb_k} = 1$ and $\theta_{ki} = 0$ for $i \neq b_k$.
- Constraints (L.2) and (L.3) together ensure incentive compatibility: $s_k = R_{k,b_k} - p_{b_k} = \max_{i \in F}\{R_{k,i} - p_i\}$.
- Only bundles in F (predicted or assigned) have price variables, reducing dimensionality.
- The model is a pure LP without integer decision variables, in contrast to the MILP formulations.
- The optimal value of the LP formulation is a lower bound of the corresponding MILP with possible bundle set $\mathfrak{F}$. That is, the optimal value of this LP is less than or equal to that of $HM(\mathfrak{F})$.

A.3 Modified Price sub-additivity Constraints

Our mixed bundling follows Hanson and Martin [1990]'s formulation in terms of decision variables, objective, and most constraints (consumer surplus, single price schedule, etc.). The only difference lies in the treatment of price sub-additivity. To enforce the general K-way Cover sub-additivity ($S_{C,K}$) proposed by Hanson and Martin [1990], we introduce a more parsimonious set of constraints. Specifically, we only enforce sub-additivity on 2-way partitions ($S_{P,2}$), which significantly reduces

the number of price sub-additivity constraints while being sufficient to guarantee the general condition. The equivalence between these two are proved in Appendix B.

Definition A.1 (K-way Cover sub-additivity — Statement $S_{C,K}$). For any bundle B and any (potentially overlapping) **cover** by $K \geq 2$ sub-bundles $\{B_1, \ldots, B_K\}$, the following holds:

$$p(B) \leq \sum_{j=1}^{K} p(B_j).$$

Definition A.2 (2-way Partition sub-additivity — Statement $S_{P,2}$). For any bundle B and any **disjoint** partition into two sub-bundles $\{B_1, B_2\}$, the following holds:

$$p(B) \leq p(B_1) + p(B_2).$$

B Proof of Equivalence Between Price sub-additivity Constraints

B.1 Definitions

Let $\mathcal{U}$ be the set containing all products, and let $p : \mathcal{P}(\mathcal{U}) \to \mathbb{R}_{\geq 0}$ be the price function, where $\mathcal{P}(\mathcal{U})$ denotes the power set of $\mathcal{U}$.

Definition B.1 (Price Monotonicity — Statement $\mathcal{M}$). For any two bundles A and B, if $A \subseteq B$, then $p(A) \leq p(B)$.

This assumption is justified by the property of free disposal. Under this setting, customers can discard unwanted items at no cost. Consequently, if a superset bundle B were priced lower than its subset A (i.e., $p(B) < p(A)$ for $A \subseteq B$), rational customers desiring A would simply purchase B, discard the items in $B \setminus A$, and effectively obtain A at a lower price. Thus, to preclude such arbitrage opportunities, optimal prices must be non-decreasing with respect to set inclusion.

Definition B.2 (K-way Partition sub-additivity — Statement $S_{P,K}$). For any bundle B and any **disjoint** partition into $K \geq 2$ sub-bundles $\{B_1, \ldots, B_K\}$, the following holds:

$$p(B) \leq \sum_{j=1}^{K} p(B_j).$$

B.2 Theorem

Proposition B.3. *Statement $S_{P,2}$ and Statement $S_{P,K}$ are equivalent.*

Proof. We prove the proposition in both directions.

Direction 1: $S_{P,K} \implies S_{P,2}$ This is true by definition. $S_{P,2}$ is a special case of $S_{P,K}$ where we choose the number of partitions $K = 2$.

Direction 2: $S_{P,2} \implies S_{P,K}$ We use mathematical induction on the number of partitions, k. The base case ($k = 2$) is Statement $S_{P,2}$, which is assumed to be true. Assume the statement holds for k partitions (Inductive Hypothesis). We prove for $K = k + 1$. Let B be partitioned into $\{B_1, \ldots, B_{k+1}\}$. Let $B' = \bigcup_{j=1}^{k} B_j$. Then $B = B' \cup B_{k+1}$ is a 2-way disjoint partition. By $S_{P,2}$:

$$p(B) \leq p(B') + p(B_{k+1}). \tag{40}$$

By the inductive hypothesis, $p(B') \leq \sum_{j=1}^{k} p(B_j)$. Substituting this yields:

$$p(B) \leq \sum_{j=1}^{k+1} p(B_j).$$

Thus, $S_{P,2} \implies S_{P,K}$. □

Proposition B.4. *Assuming Price Monotonicity ($\mathcal{M}$), Statement $S_{P,K}$ and Statement $S_{C,K}$ are equivalent.*

Proof. We prove the proposition in both directions, assuming $\mathcal{M}$ holds.

Direction 1: $S_{C,K} \implies S_{P,K}$. A disjoint partition is a special case of a cover. Thus, if the rule holds for any cover, it must hold for any disjoint partition.

Direction 2: $(S_{P,K} \wedge \mathcal{M}) \implies S_{C,K}$. We show that $(S_{P,2} \wedge \mathcal{M}) \implies S_{C,2}$ (the binary case). Let $B = B_1 \cup B_2$ be a cover. We can write B as a disjoint partition: $B = (B_1 \setminus B_2) \cup B_2$. By $S_{P,2}$:

$$p(B_1 \cup B_2) \leq p(B_1 \setminus B_2) + p(B_2). \tag{41}$$

Since $(B_1 \setminus B_2) \subseteq B_1$, by Monotonicity ($\mathcal{M}$), we have $p(B_1 \setminus B_2) \leq p(B_1)$. Substituting this gives $p(B_1 \cup B_2) \leq p(B_1) + p(B_2)$, which proves $S_{C,2}$.

Since $S_{P,K} \iff S_{P,2}$ (from Prop. 1) and $S_{C,K} \iff S_{C,2}$ (by induction), it follows that $(S_{P,K} \wedge \mathcal{M}) \implies S_{C,K}$. □

Theorem B.5. *Assuming Price Monotonicity ($\mathcal{M}$), the 2-way Partition sub-additivity ($S_{P,2}$) is equivalent to the K-way Cover sub-additivity ($S_{C,K}$).*

Proof. From Proposition 1, $S_{P,2} \iff S_{P,K}$, and from Proposition 2 under Price Monotonicity $\mathcal{M}$, $S_{P,K} \iff S_{C,K}$. Hence $S_{P,2} \iff S_{C,K}$. □

C Cutoff Sensitivity Analysis

This appendix examines the sensitivity of our pruning strategies to the cutoff threshold used to binarize the GCN-predicted purchase probabilities. The cutoff determines which products are retained in the candidate space and therefore directly affects both solution quality and computational efficiency. We vary the cutoff in $\{0.1, 0.2, \ldots, 0.9\}$ and evaluate both FCP and PCP under identical experimental settings, reporting the revenue ratio and time ratio averaged over multiple instances.

Revenue–Time Trade-off. Sensitivity results based on the revenue ratio and time ratio jointly indicate that cutoff = 0.5 provides a well-balanced operating point for both FCP and PCP. For FCP, the revenue ratio increases monotonically as the cutoff rises, but the marginal improvement beyond 0.5 is limited. For PCP, the revenue ratio is highly stable over a wide range of cutoffs (approximately 0.99 for cutoff $\leq$ 0.6), whereas the time ratio decreases substantially as the cutoff increases. In particular, moving from low cutoffs to 0.5 already yields a pronounced runtime reduction. Taken together, these results show that cutoff = 0.5 lies in a revenue-saturated region for both strategies, while still capturing most of the achievable computational gains, making it a natural and robust trade-off point.

Accuracy–Recall Trade-off. The cutoff threshold also governs a fundamental trade-off between prediction accuracy and recall. As the cutoff increases, precision and overall accuracy improve due to more aggressive pruning, but recall decreases and the false negative rate (FNR) rises, reflecting an increasing risk of excluding products that appear in the optimal bundle. This effect becomes noticeably stronger beyond cutoff = 0.5, where recall starts to decline at a faster rate. Since false negatives are particularly harmful in bundle pricing, cutoff = 0.5 strikes a conservative balance: it substantially reduces false positives while keeping FNR at a relatively low level. This further supports the choice of cutoff = 0.5 as a safe default that balances pruning aggressiveness with coverage of revenue-critical products.

Conclusion. Overall, the cutoff sensitivity analysis demonstrates that our proposed strategies are robust to the choice of threshold within a broad range. Selecting cutoff = 0.5 consistently achieves

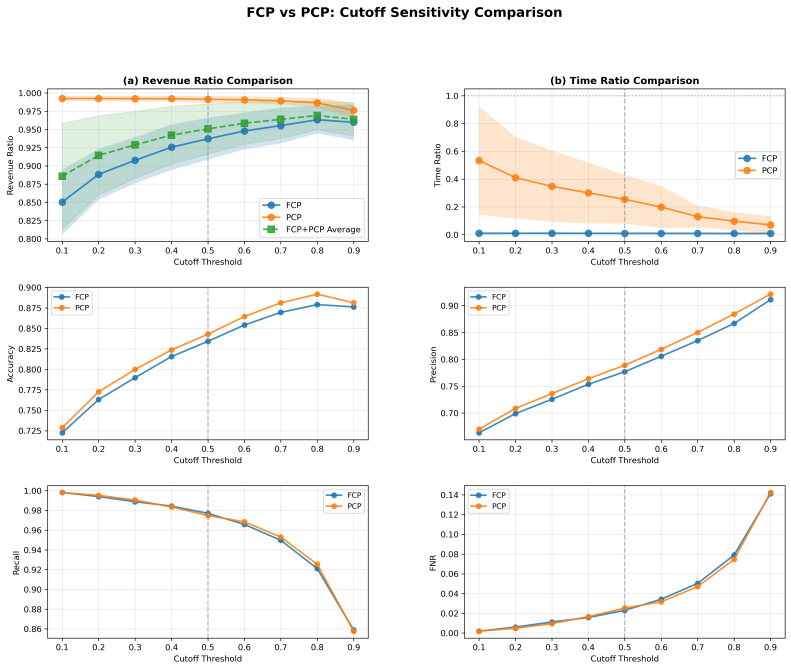

Fig. 6. Sensitivity of revenue and runtime to the cutoff threshold for FCP and PCP.

a near-saturated revenue level for both FCP and PCP, while delivering substantial computational savings and maintaining a conservative balance between precision and recall.

D Sensitivity Analysis of Candidate Size K

To justify our choice of $K = \lceil \sqrt{m} \rceil$ in the Global Top-K strategy, we conduct a controlled sensitivity analysis comparing multiple candidate-size heuristics across datasets with varying numbers of customer segments ($m \in \{10, 20, 30\}$) and products ($n \in \{10, 15, 20, 25\}$). The evaluated strategies include:

- **Exhaustive ($2m$):** Segment-wise linear expansion with complexity $O(m)$.
- **Constant-K:** Fixed thresholds ($K = 5, 10$) independent of problem scale.
- **Sqrt-m (Proposed):** Adaptive candidate size $K = \lceil \sqrt{m} \rceil$.
- **Sqrt-mn:** Joint segment–product scaling $K = \lceil \sqrt{mn} \rceil$.

Figure 7 reports both Pareto comparisons (revenue vs. runtime) and efficiency (revenue per unit time). The Pareto plots (left panels) indicate that the *Sqrt-m* strategy consistently traces a curve that lies further to the left, meaning it achieves comparable revenue with lower runtime, reflecting an efficient exploration–exploitation balance with minimal unnecessary candidate expansion. Meanwhile, the efficiency plots (right panels) show that *Sqrt-m* attains the highest or near-highest revenue per unit time across all configurations, and this advantage remains stable as the problem scale increases. Together, these results identify $K = \lceil \sqrt{m} \rceil$ as a well-balanced and robust choice.

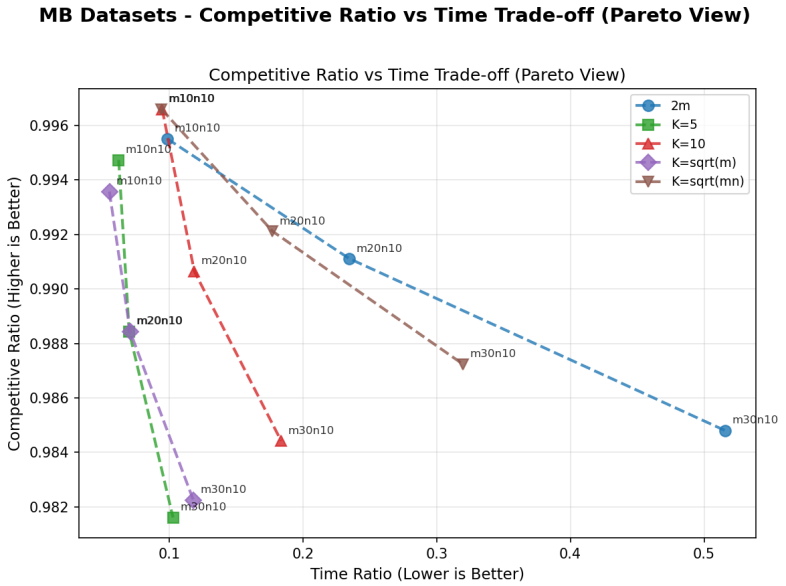

Fig. 7. Sensitivity analysis of candidate size K across BSP and MB datasets. Left panels show relative revenue and runtime differences normalized to the *Sqrt-m* baseline. Right panels report efficiency measured as revenue per unit time.

E Consistency Analysis of LP and MILP Improvements

To address concerns regarding the consistency between the Linear Programming (LP) relaxation and the original Mixed Integer Linear Programming (MILP) problem during the local search process, we conducted a comprehensive tracking experiment. The goal was to verify whether improvements in the LP objective (Z_{LP}) effectively translate into gains in the MILP objective (Z_{MILP}).

E.1 Experimental Setup

We selected 60 instances across varying problem sizes (ranging from $m = 10$ to $m = 30$ and varying bundle sizes n) from our testbed. For each instance, we executed the proposed Local Search algorithm. At every iteration where an LP-guided improvement was identified (i.e., $\Delta Z_{LP} > \epsilon$, with $\epsilon = 10^{-6}$), we explicitly calculated the corresponding exact MILP objective value to verify the move's quality.

E.2 Quantitative Analysis of Improvement Translation

We define an **"Effective Translation"** as a move where an increase in the LP lower bound leads to a strict increase in the MILP objective ($\Delta Z_{MILP} > 0$). In numerical results, the LP-guided strategy demonstrates exceptional stability: Out of **610** total accepted moves across all datasets, **595** resulted in immediate MILP improvements. The **Global Translation Rate** is **97.5%**, indicating that the LP relaxation serves as a highly reliable proxy for the integer problem in the neighborhood structure.

E.3 Visual Trajectory

Figure 8 illustrates the optimization trajectory for a representative sample. The LP and MILP objective values exhibit a synchronized upward trend, confirming that optimizing the relaxed problem efficiently drives the integer solution quality.

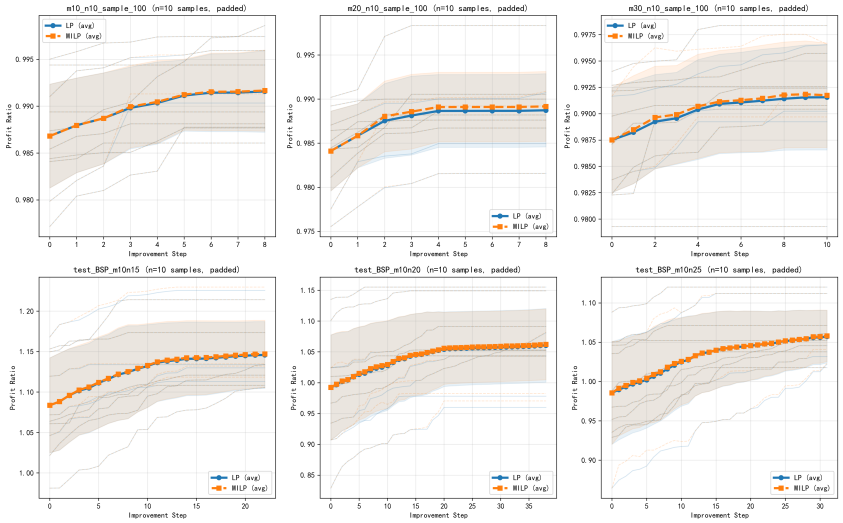

Fig. 8. Trajectory comparison of LP and MILP objective values during Local Search iterations. The synchronized steps indicate that LP relaxation effectively guides the search towards high-quality integer solutions.

F Pseudocode for the Local Search Strategies

Algorithm 2 Fixed Cutoff Pruning (FCP) Policy

Input: Predicted probability matrix $\mathbb{P} \in [0,1]^{m \times n}$, parameters $(\mathbf{U}, \mathbf{c}^{\mathbf{u}}, \mathbf{c}^{\mathbf{s}}, \boldsymbol{\alpha})$, cutoff $\tau = 0.5$
Output: Optimal prices $\mathbf{p}^*$ and assignment $\boldsymbol{\theta}^*$
Initialize candidate set $\mathfrak{F}^{\mathrm{FCP}} \leftarrow \emptyset$
for $k = 1$ to m **do**
  $S_k \leftarrow \{j \in [n] \mid \mathbb{P}_{kj} \geq \tau\}$ ▷ Identify high-probability products
  if $S_k = \emptyset$ **then** ▷ Avoid empty bundle by selecting max probability product
    $j^* \leftarrow \arg\max_{j \in [n]} \mathbb{P}_{kj}$
    $B_k^{\mathrm{FCP}} \leftarrow \{j^*\}$
  else
    $B_k^{\mathrm{FCP}} \leftarrow S_k$
  end if
  $\mathfrak{F}^{\mathrm{FCP}} \leftarrow \mathfrak{F}^{\mathrm{FCP}} \cup \{B_k^{\mathrm{FCP}}\}$
end for
Solve $HM(\mathfrak{F}^{\mathrm{FCP}})$ to obtain optimal solution
return $\mathbf{p}^*, \boldsymbol{\theta}^*$

Algorithm 3 Progressive Cutoff Pruning (PCP) Policy

Input: Predicted probability matrix $\mathbb{P} \in [0,1]^{m \times n}$, parameters $(\mathbf{U}, \mathbf{c}^{\mathbf{u}}, \mathbf{c}^{\mathbf{s}}, \boldsymbol{\alpha})$, cutoff $\tau = 0.5$
Output: Optimal prices $\mathbf{p}^*$ and assignment $\boldsymbol{\theta}^*$
Initialize candidate set $\mathfrak{F}^{\mathrm{PCP}} \leftarrow \emptyset$, constraints $C_{\mathrm{chain}} \leftarrow \emptyset$
for $k = 1$ to m **do**
  $U_k \leftarrow \{j \in [n] \mid \mathbb{P}_{kj} \geq \tau\}$ ▷ Filter products
  Sort U_k by descending $\mathbb{P}_{kj}$ to obtain sequence $(j_{(1)}, j_{(2)}, \ldots, j_{(|U_k|)})$
  Initialize prefix chain $B_{k,0}^{\mathrm{PCP}} \leftarrow \emptyset$
  for $i = 1$ to $|U_k|$ **do**
    $B_{k,i}^{\mathrm{PCP}} \leftarrow B_{k,i-1}^{\mathrm{PCP}} \cup \{j_{(i)}\}$ ▷ Progressive construction
    $\mathfrak{F}^{\mathrm{PCP}} \leftarrow \mathfrak{F}^{\mathrm{PCP}} \cup \{B_{k,i}^{\mathrm{PCP}}\}$
    Add inequality $p(B_{k,i-1}^{\mathrm{PCP}}) \leq p(B_{k,i}^{\mathrm{PCP}})$ to C_{chain} ▷ Monotonicity constraint
  end for
end for
Solve $HM(\mathfrak{F}^{\mathrm{PCP}})$ subject to C_{chain} (replacing intra-segment constraints)
return $\mathbf{p}^*, \boldsymbol{\theta}^*$

Algorithm 4 Adaptive Global Local Search (Global Top-K)

Input: Initial solution $x \in \{0,1\}^{m \times n}$, Probability matrix $\mathbb{P}$, MaxIter
Output: Optimized solution x^*
Initialize: $x^* \leftarrow x$, $rev^* \leftarrow$ LP-Eval(x^*)
Parameter: $K \leftarrow \lceil \sqrt{m} \rceil$ ▷ Adaptive sublinear neighborhood
$iter \leftarrow 0$
while $iter <$ MaxIter **do**
  $iter \leftarrow iter + 1$, $improve \leftarrow$ FALSE
  Step 1: Identify Candidates Globally
  Let $\mathcal{U} = \{(k,j) : x^*_{kj} = 0\}$ be the set of unselected items
  Let $\mathcal{S} = \{(k,j) : x^*_{kj} = 1\}$ be the set of selected items
  $C_{\text{add}} \leftarrow \arg\text{top}_K\{\mathbb{P}_{kj} \mid (k,j) \in \mathcal{U}\}$ ▷ Highest probability adds
  $C_{\text{drop}} \leftarrow \arg\text{top}_K\{1 - \mathbb{P}_{kj} \mid (k,j) \in \mathcal{S}\}$ ▷ Lowest probability drops
  Step 2: Mix and Sort
  $C \leftarrow$ SortDescending$(C_{\text{add}} \cup C_{\text{drop}})$ based on scores
  Step 3: Sequential Evaluation
  for move $\mu \in C$ **do**
    $y \leftarrow$ ApplyMove(x^*, μ)
    $(feas, rev) \leftarrow$ LP-Eval(y)
    if $feas$ **and** $rev > rev^* + \epsilon$ **then**
      $x^* \leftarrow y$, $rev^* \leftarrow rev$
      $improve \leftarrow$ TRUE
      break ▷ Greedy accept & restart iteration
    end if
  end for
  if not $improve$ **then**
    break ▷ Converged
  end if
end while
return x^*

We denote by $\arg\text{top}_K(\mathcal{X})$ the operator that returns the set of K elements from $\mathcal{X}$ with the highest values. Formally, given a set of candidate moves and their associated scores, this operator selects the subset of size K that maximizes the scores, breaking ties arbitrarily.

G Seed Performance

In this section, we present the results under 10 independent trials as the random seed ranging from 1 to 10.

H Out of Distribution Performance

In this section, we demonstrate the robustness of our model by testing out-of-distribution performance across utility families $f(\cdot)$ and distribution shifts of Base Utility u_{kj}. We evaluate the model directly on test instances with significantly different underlying utilities or distributions without any re-training.

H.1 Robustness to Utility Function Shifts

We first examine the impact of varying the utility function $f(\cdot)$ in Table H.1.

Table G.1. Performance metrics across different random seeds for $m_{test} = 10, n_{test} = 10$

Seed	$CR_{FCP,MB}$	$TR_{FCP,MB}$	Time (s)
1	0.974	0.007	0.048
2	0.979	0.007	0.047
3	0.968	0.007	0.046
4	0.972	0.007	0.046
5	0.974	0.007	0.047
6	0.977	0.007	0.046
7	0.965	0.007	0.049
8	0.979	0.007	0.047
9	0.968	0.007	0.048
10	0.977	0.007	0.045

Logarithmic Utility ($f(x) = \log(1 + x)$). This function represents a slower rate of diminishing returns compared to the training function. As shown in the results, the model exhibits high stability, with the FCP strategy achieving a 96.3% competitive ratio and PCP further improving it to 98.4%.

Cubic Root Utility ($f(x) = x^{\frac{1}{3}}$). This utility introduces more aggressive diminishing returns, creating a more challenging combinatorial landscape. While the FCP strategy sees a slight performance dip to 94.3%, the PCP strategy successfully mitigates this difficulty by exploring a broader candidate space, restoring the optimality to 98.6%.

H.2 Robustness to Distribution Shifts

We further evaluate robustness against shifts in the base utility u_{kj} in Table H.2, using Beta distributions to model distinct customer preference patterns.

Concentrated Preferences (Beta$(5, 5)$). This unimodal distribution represents highly concentrated customer preferences with low variance, reducing the distinction between products. Despite this homogeneity, our model demonstrates strong robustness, with FCP achieving 97.5% optimality and PCP reaching 98.5%.

Polarized Preferences (Beta$(0.5, 0.5)$). This U-shaped distribution represents polarized preferences with high variance, where customers tend to either strongly like or dislike products. In this setting, our framework performs exceptionally well, with FCP and PCP attaining 98.0% and 99.0% optimality, respectively.

In summary, our framework demonstrates strong robustness across diverse utility shapes and preference distributions, achieving consistently high optimality (over 94% for FCP and over 98% for PCP) with orders-of-magnitude speedup compared to mixed bundling, even on out-of-distribution instances.

Table H.1. Performance robustness under utility function shift ($m = 10, n = 10$).

Scenario	**Strategy**	$CR_{\cdot,MB}$ **(std)**	**Time (s)**	$TR_{\cdot,MB}$
$f(x) = \log(1 + x)$	FCP	0.963 (0.024)	0.054	0.007
	PCP	0.984 (0.014)	0.338	0.042
$f(x) = x^{\frac{1}{3}}$	FCP	0.943 (0.021)	0.055	0.008
	PCP	0.986 (0.007)	0.280	0.039

Table H.2. Performance robustness under distribution shift ($m = 10, n = 10$).

Scenario	**Strategy**	$CR_{\cdot,MB}$ **(std)**	**Time (s)**	$TR_{\cdot,MB}$
$u_{kj} \sim \text{Beta}(5, 5)$	FCP	0.975 (0.014)	0.054	0.009
	PCP	0.985 (0.012)	0.397	0.062
$u_{kj} \sim \text{Beta}(0.5, 0.5)$	FCP	0.980 (0.019)	0.046	0.007
	PCP	0.990 (0.010)	0.210	0.033